

数据与算法 - 第7讲

图 (一)

冯伟 (fengwei@tsinghua.edu.cn)

2024年10月29日





提纲

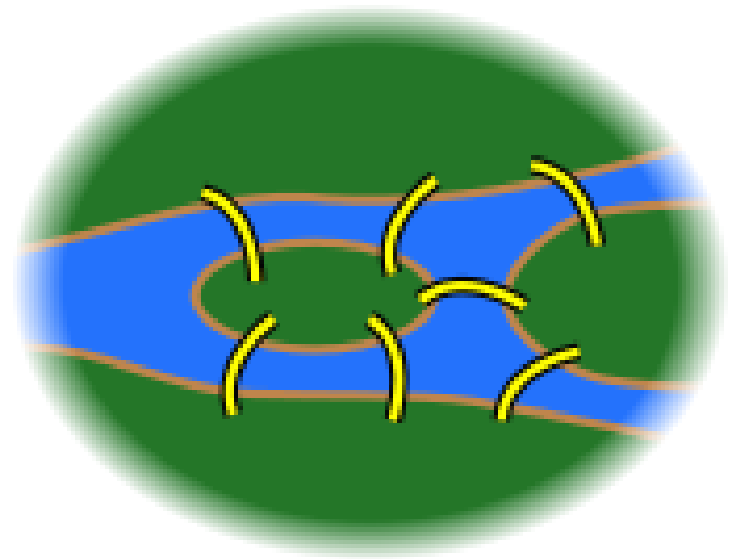
- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树
- 最短路径问题（下一讲）



- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树

图的例子

- 10个人参加一个聚会，之前有些人互相认识，有些不认识。组织者统计后，每人在聚会前认识的人数分别为3, 3, 3, 3, 5, 6, 6, 6, 6, 6。请问统计是否有错？
- 一笔画问题：日、中、田、目是否可以一笔画出？
- 科尼斯堡七桥问题：有一个小岛，有七座桥将这个小岛与河的两岸以及交叉口之间的陆地连接起来。有没有一种方法连续地穿过镇上的这七座桥，同时又不重复走过每座桥？





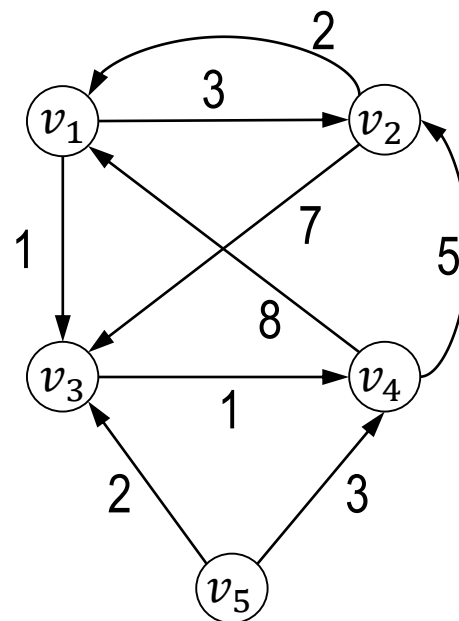
图的基本概念

- 在图中并不存在特殊的根结点，每个结点都可能存在多个相邻的结点
- 图比线性结构和树结构更为复杂、但具有更强的描述能力
- 图可用二元组 $G = (V, E)$ 表示
 - V 是一个有限集合；
 - E 是 V 中元素的有序(或无序)二元组 $E = \{\langle v, w \rangle | v, w \in V, v \neq w\}$
- V 的元素称为顶点， E 的元素 $\langle v, w \rangle$ 称为连接 v 和 w 的边，如果 $\langle v, w \rangle$ 是图的一条边，则称 v 与 w 互为邻接顶点
- V 称为图的顶点集， E 称为图的边集， $\|V\|$ 和 $\|E\|$ 分别表示 V 和 E 中元素的个数

看待一个图结构有从顶点出发和从边出发两种不同视角

图 - 定义

- 权：图的边具有与它相关的数
- 带权图也称为网络
- 简单图（本课程关注简单图）
 - 图中不存在闭环，即不存在从某个顶点到自身的边
 - 图中不存在多重边，即起点和终点相同的边不能多于一条



自是鼻子
“向内的自己”



己是绳子
“对外的自己”

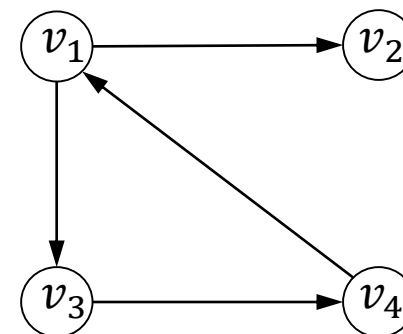
简单图关注
一阶邻接关系



图 - 定义

- 如果边集 E 的元素 $\langle v, w \rangle$ 为有序二元组，则称 G 为**有向图**， v 称为起点， w 称为终点

有向图 G_1

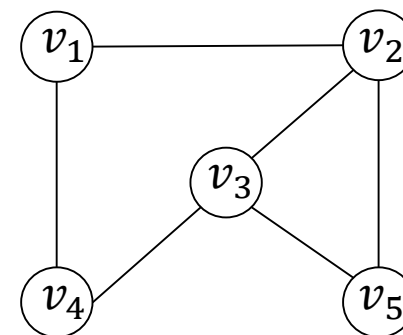


$$G_1 = \langle V_1, E_1 \rangle, \quad V_1 = \{v_1, v_2, v_3, v_4\}$$

$$E_1 = \{\langle v_1, v_2 \rangle, \langle v_1, v_3 \rangle, \langle v_3, v_4 \rangle, \langle v_4, v_1 \rangle\}$$

- 如果边集 E 的元素 $\langle v, w \rangle$ 是无序的，则称 G 为**无向图**

无向图 G_2

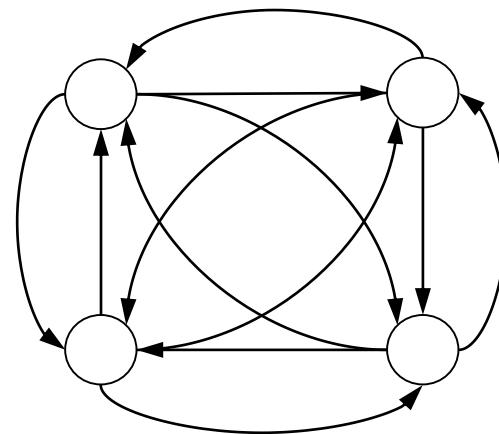


$$G_2 = \langle V_2, E_2 \rangle, \quad V_2 = \{v_1, v_2, v_3, v_4, v_5\}$$

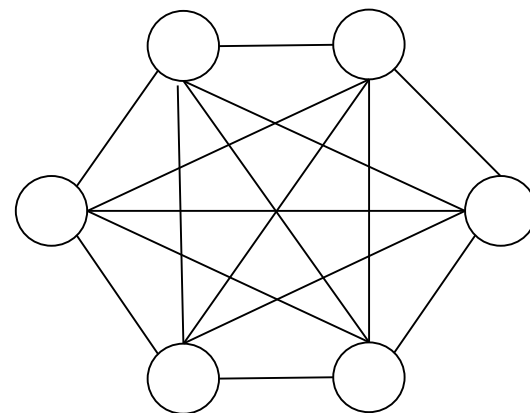
$$E_2 = \{\langle v_1, v_2 \rangle, \langle v_1, v_4 \rangle, \langle v_2, v_3 \rangle, \langle v_2, v_5 \rangle, \langle v_3, v_4 \rangle, \langle v_3, v_5 \rangle\}$$

图 - 定义

- 对图 G , 令 $n = \|V\|$, $e = \|E\|$
- $e = n \times (n - 1)$ 的有向图称为**有向完全图**
- $e = n \times (n - 1)/2$ 的无向图称为**无向完全图**
- n 个顶点的无向图共有 $2^{n(n-1)/2}$
- 边数很少的图称为**稀疏图**
- 边数很多的图称为**稠密图**



有向完全图示例



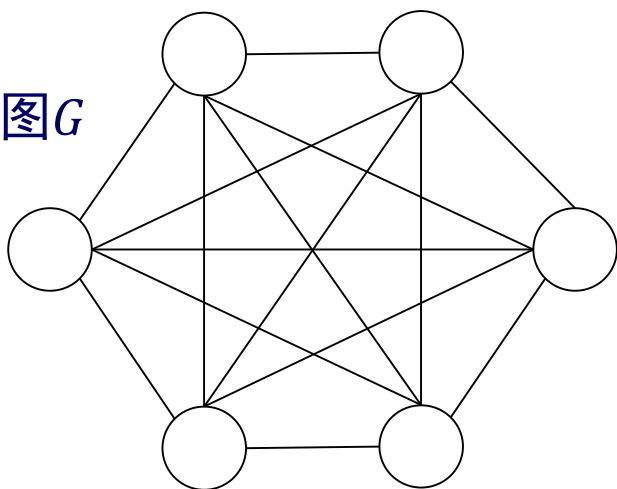
无向完全图示例

图 - 定义

设有两个图 $G = \langle V, E \rangle$ 和 $G^* = \langle V^*, E^* \rangle$ 。若 $V^* \subseteq V$ 且 $E^* \subseteq E$, 则称图 G^* 是图 G 的子图

- 一个图可能有很多子图
- 每一个图都是它自身的子图, 空图是任何一个图的子图
- 具有 n 个顶点的图是 n 个顶点的完全图的子图

完全图 G



子图 G^*

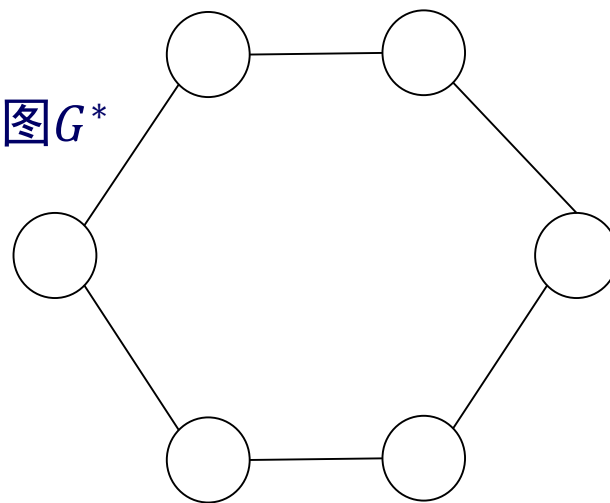
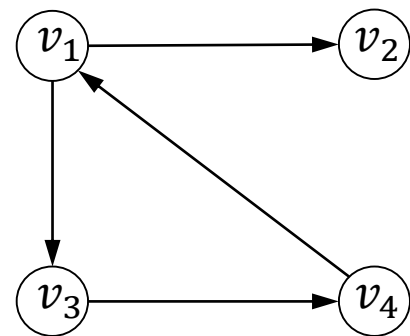




图 - 定义

- 一个顶点 v 的度是与它相关联的边的条数, 记作 $TD(v)$
- 有向图中顶点 v 的入度是以 v 为终点的有向边的条数, 记作 $ID(v)$
- 有向图中顶点 v 的出度是以 v 为起点的有向边的条数, 记作 $OD(v)$
- 有向图中顶点的度等于该顶点的入度与出度之和
- 有向图中的所有顶点的入度之和等于所有顶点的出度之和

有向图 G_1



无向图 G_2

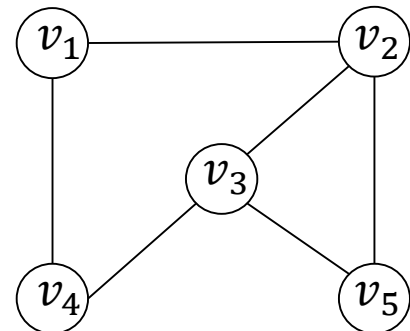
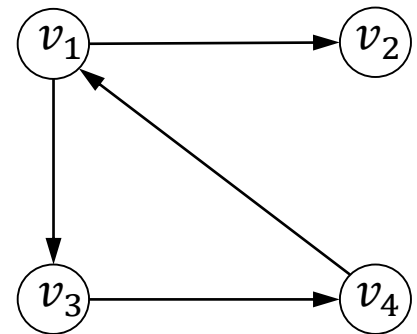




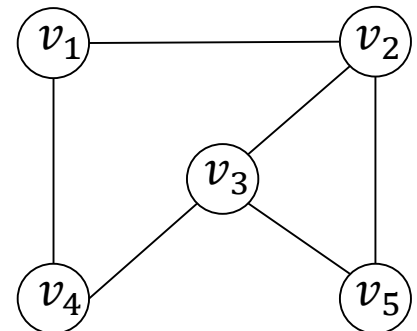
图 - 定义

- n 个顶点的无向图中某个顶点的度最多为 $n - 1$
- n 个顶点的有向图中某个顶点的度最多为 $2(n - 1)$
- **握手定理**：图中所有顶点的度之和等于边数之和的2倍
- 推论：有向图所有顶点的入度之和等于边数之和，所有顶点的出度之和也等于边数之和
- 度为奇数的顶点称为奇度顶点，简称**奇点**，度为偶数的顶点称为偶度顶点，简称**偶点**
- 任意一个图一定有偶数个(或0个)奇点

有向图 G_1

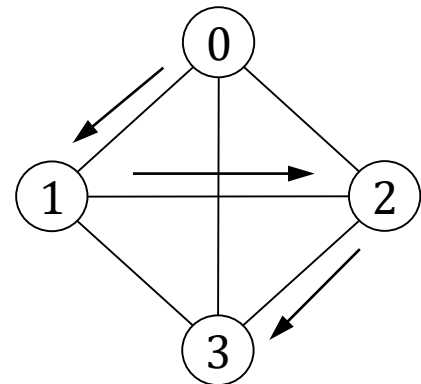


无向图 G_2

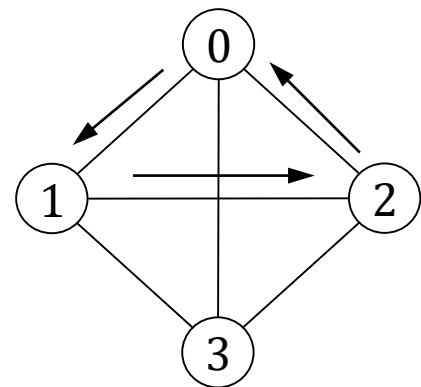


路径与回路

- 在图 $G = \langle V, E \rangle$ 中, 从顶点 v 到 v^* 的路径是一个顶点序列 $(v_{i_0} = v, v_{i_1}, v_{i_2}, \dots, v_{i_m} = v^*)$, 其中 $\langle v_{i_{j-1}}, v_{i_j} \rangle \in E, 1 \leq j \leq m$
- 路径长度是路径上边的数目
- 路径序列中没有重复出现的顶点称为简单路径
- 路径的起点和终点相同($v = v^*$)称为回路或环
- 除了 v 之外, 其余顶点不重复出现的回路称为简单回路



简单路径



回路

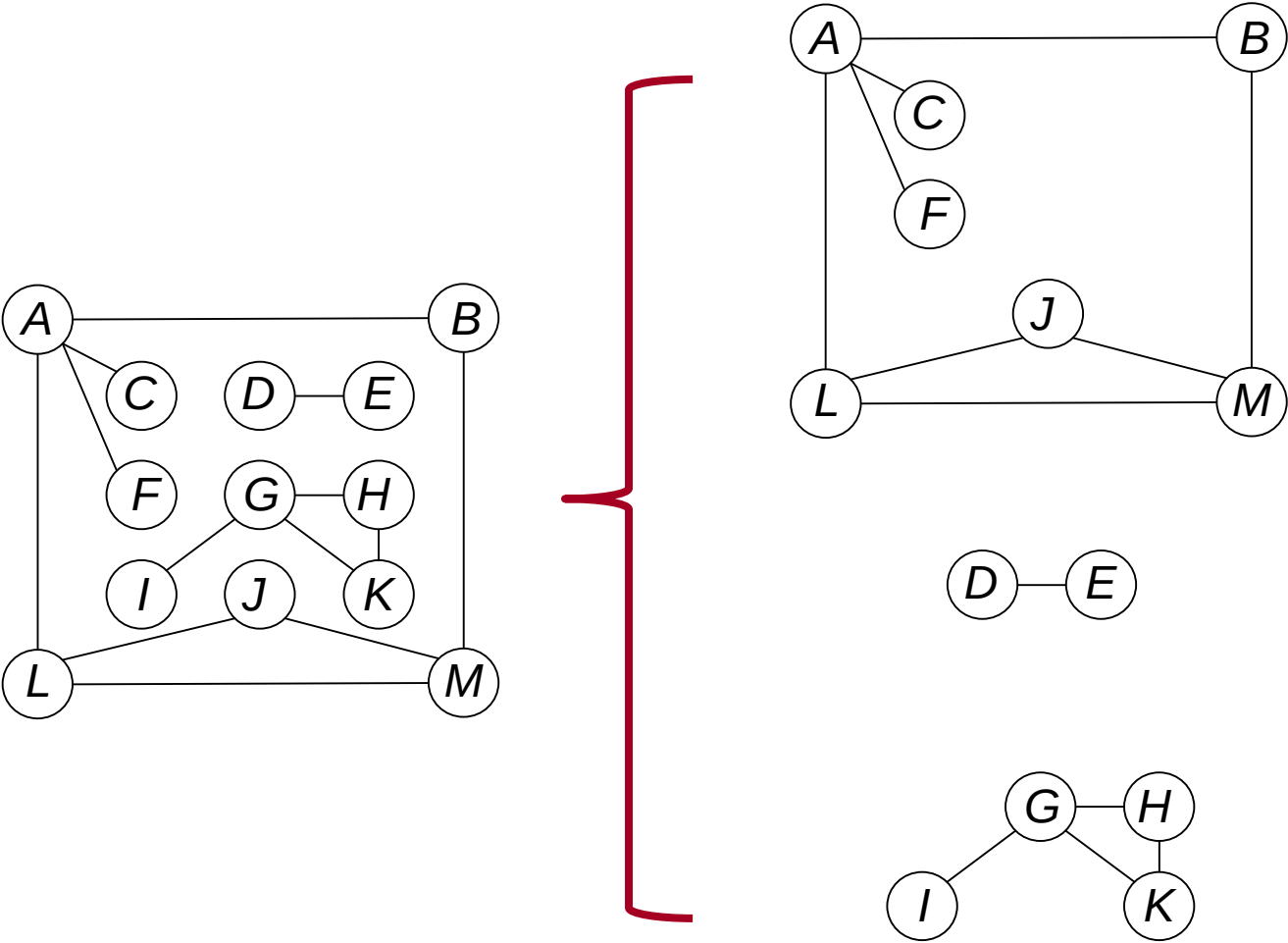


连通与连通分量

- 在无向图中，如果从顶点 v 到顶点 w 有路径，则称 v 和 w 是连通的
- 在无向图中，顶点之间的连通关系是等价关系
- 如果对于图中任意两个顶点 $v_i, v_j \in V$ ， v_i 和 v_j 都是连通的，则称 G 是**连通图**
- **极大连通子图**：图 G 中任何不属于这个子图的顶点与这个子图中的任何一个顶点都是不连通的
- 无向图的极大连通子图称为**连通分量**



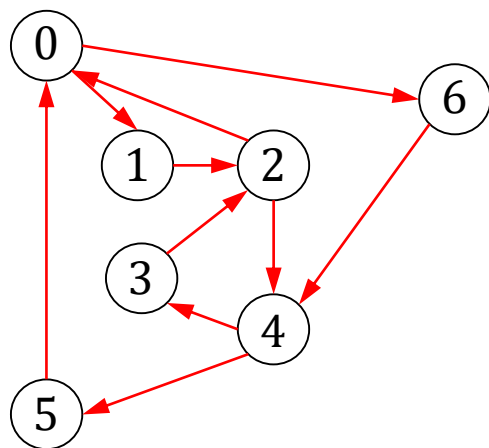
连通与连通分量



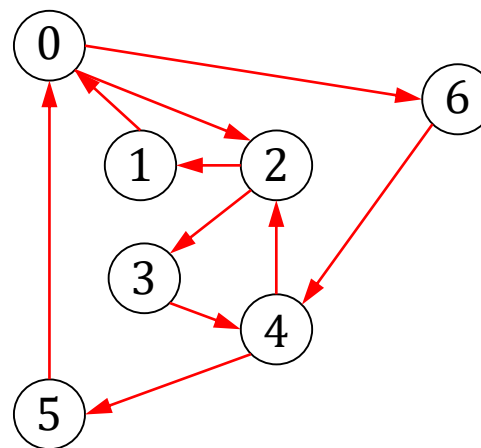


欧拉路径

- 欧拉（Euler）路径是遍历图中每条边且只访问一次的路径
- 终点回到起点的欧拉路径是欧拉回路



0-1-2-0-6-4-3-2-4-5-0

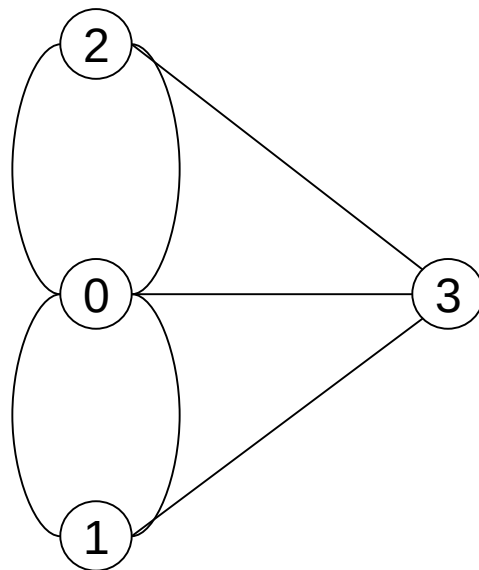
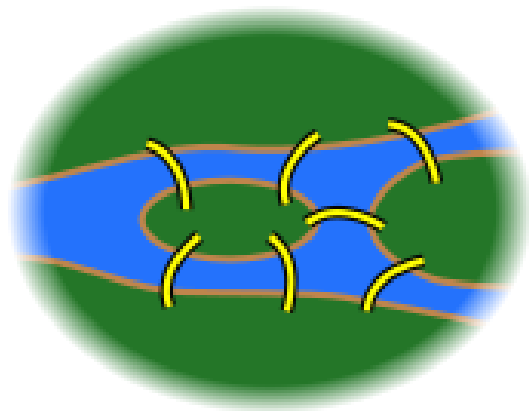


0-6-4-2-3-4-5-0-2-1-0

欧拉路径中顶点可以重复访问，可能不唯一

欧拉路径

- 一个图有欧拉回路的充分必要条件：图是连通的且每一个顶点的度都是偶数
- 一个图有欧拉路径的充分必要条件：图是连通的且仅有两个顶点的度是奇数，或者没有顶点的度是奇数
- 因此，七桥问题是无解的





图的抽象数据类型

ADT Graph {

数据对象： V 是具有相同特性的数据元素的集合，称为顶点集

数据关系： $R = \{\langle v, w \rangle | v, w \in V, v \neq w\}$, $\langle v, w \rangle$ 表示连接 v 和 w 的边}

基本操作：

InitGraph(&G, n, d);

操作结果：按顶点个数 n 和标志 d 构造图 G

DestroyGraph(&G);

初始条件：图 G 已存在

操作结果：销毁图 G



图的抽象数据类型

IsDirected();

初始条件：图G已存在

操作结果：如果图G是有向图，返回TRUE，否则返回FALSE

Vcnt(&G);

初始条件：图G已存在

操作结果：返回图G的顶点数

Ecnt(&G);

初始条件：图G已存在

操作结果：返回图G的边数



图的抽象数据类型

$\text{FirstAdjVex}(G, v);$

初始条件：图G已存在

操作结果：返回v的第一个邻接顶点。若该顶点在G中没有邻接顶点，则返回“空”

$\text{NextAdjVex}(G, v, w);$

初始条件：图G已存在

操作结果：返回v的（相对于w的）下一个邻接顶点

若w是v的最后一个邻接顶点，则返回“空”

逐顶点视角的基本操作

$\text{Insert}(\&G, v, w);$

初始条件：图G已存在

操作结果：插入边 $\langle v, w \rangle$



图的抽象数据类型

Delete(&G, v, w);

初始条件：图G已存在

操作结果：删除边 $\langle v, w \rangle$

Edge(&G, v, w);

初始条件：图G已存在

操作结果：若存在边 $\langle v, w \rangle$ ，返回TRUE，否则返回FALSE

Traverse(G, v, Visit());

初始条件：图G已存在

操作结果：从顶点v起遍历图G，并对每个顶点调用函数Visit

} ADT GRAPH



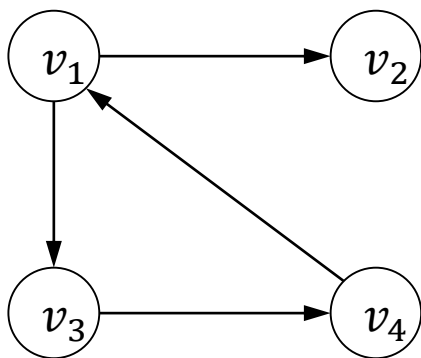
提纲

- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树

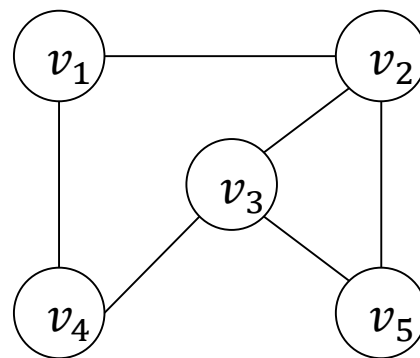


图的存储结构 - 邻接矩阵

- 用矩阵表示图中顶点之间的邻接关系
 - n 个顶点，其邻接矩阵是一个 $n \times n$ 的矩阵
 - 行表示边的起点，列表示边的终点，元素为布尔值：若在图中存在一条连接顶点 v 和 w 的边，则将 v 行， w 列的元素定义为1，否则即定义为0
- 图的邻接矩阵表示需要与 n^2 成比例的存储空间
- 与边数目无关，适用于表示稠密图



$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

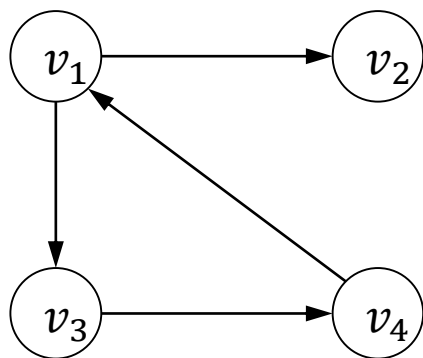


$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

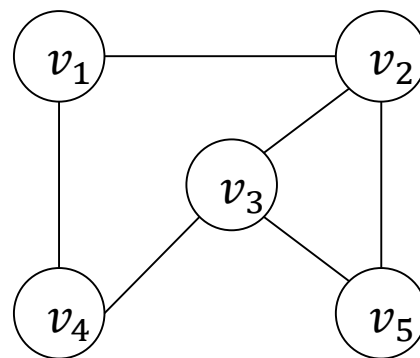


图的存储结构 - 邻接矩阵

- 无向图的邻接矩阵是对称的，有向图的邻接矩阵可能是不对称的
- 统计有向图第 i 行中1的个数可得顶点 i 的出度，统计第 j 列中1的个数可得顶点 j 的入度
- 在无向图中，统计第 i 行(列)中1的个数可得顶点 i 的度



$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

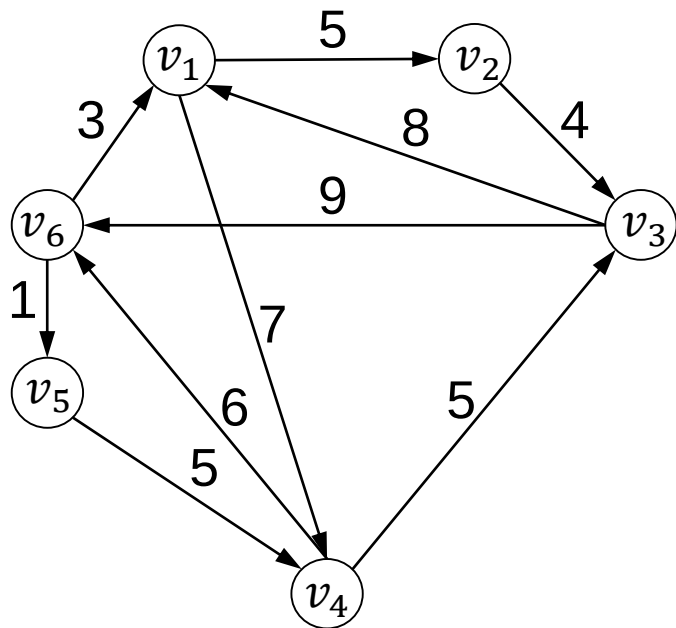


$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$



图的存储结构 - 邻接矩阵

- 如果将邻接矩阵中元素改为边的权值，邻接矩阵可以用来表示带权图
- 对角线元素仍为0，其他不存在的边，对应元素设为 ∞



0	5	∞	7	∞	∞
∞	0	4	∞	∞	∞
8	∞	0	∞	∞	9
∞	∞	5	0	∞	6
∞	∞	∞	5	0	∞
3	∞	∞	∞	1	0



图的存储结构 - 邻接矩阵

- 邻接矩阵可以用一个数组实现
- 程序实现时，除了顶点数、边数外，往往设置有向图/无向图标志、顶点访问标志（更好应对图结构的复杂性）
- 基于邻接矩阵典型操作的时间复杂度：
 - 边插入操作 $O(1)$
 - 边删除操作 $O(1)$
 - 判边存在操作 $O(1)$
 - 访问图中某个顶点的所有邻接顶点 $O(n)$
 - 依次访问所有顶点的所有邻接顶点 $O(n^2)$

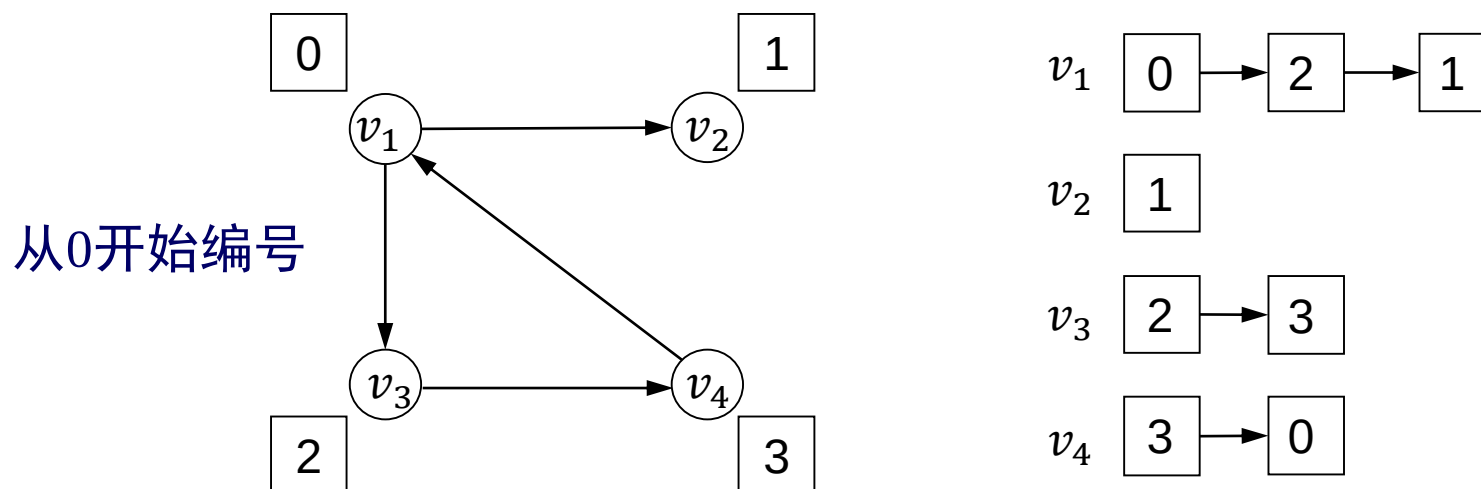


图的存储结构 - 邻接表

- 邻接矩阵表示适于稠密图
- 对于稀疏图为了改善空间效率，可以采用链式存储，即邻接表表示
- 邻接表
 - 为每个顶点建立起一个表示邻接关系的单链表，链表的各个结点表示图中存在从单链表头结点到某个邻接顶点的一条边
 - 每个链表头结点为图的顶点
 - 同一个顶点发出的边链接在同一个链表中
 - 链表结点也被称为边结点



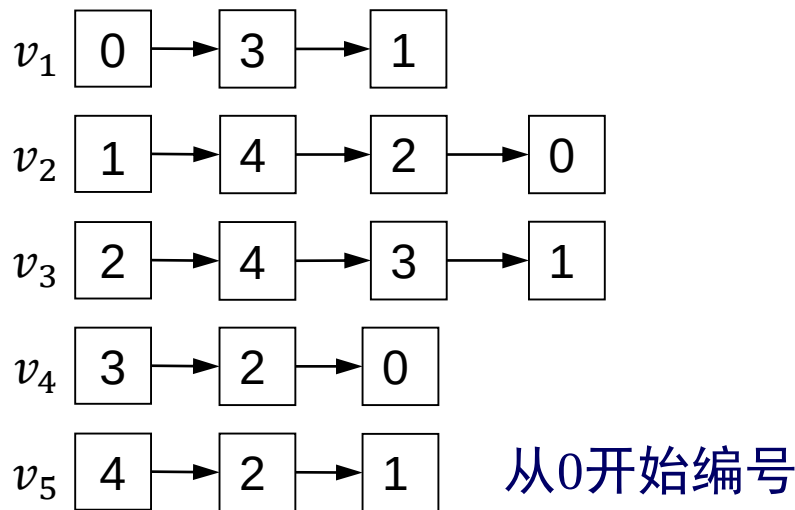
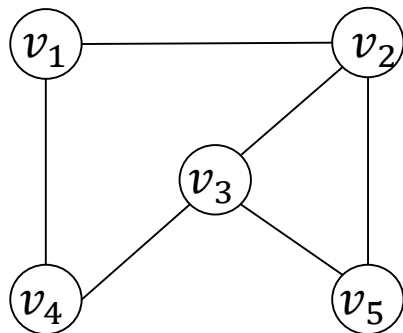
图的存储结构 - 邻接表 - 有向图



- 以某个顶点为头结点的链表长度减1（或除头结点外的边结点个数）就等于这个顶点的出度
- 有向图的邻接表表示需要 $(n + e)$ 个链表结点
- 一条边只存在于以其起点为头结点的单链表中



图的存储结构 - 邻接表 - 无向图



- 以某个顶点为头结点的链表长度减1等于顶点的度
- 一条边同时存在于其连接的顶点为头结点的两个单链表中
- 无向图的邻接表表示需要 $(n + 2e)$ 个链表结点
- 在邻接表中，各个边结点的链入顺序任意，视边结点输入次序而定，因此，图的邻接表表示不是唯一的



图的存储结构 - 邻接表

- 邻接表可以用一个链表数组实现
- 程序实现时，注意设置顶点数、边数、有向图/无向图标志、顶点访问标志等，注意链表结点的内存管理
- 基于邻接表典型操作的时间复杂度：
 - 边插入操作 $O(1)$
 - 边删除操作 $O(n)$ 【沿着指针寻找待删除的边】
 - 判边存在操作 $O(n)$
 - 访问图中某个顶点的所有邻接顶点 $O(n)$
 - 依次访问所有顶点的所有邻接顶点 $O(n + e)$ （有向图）



图的存储结构对比

- 邻接矩阵和邻接表是普遍采用的表示图的方法

	邻接矩阵	邻接表（有向图为例）
占用空间	$O(n^2)$	$O(n + e)$
初始化为空	$O(n^2)$	$O(n)$
复制	$O(n^2)$	$O(n + e)$
访问所有邻接顶点	$O(n^2)$	$O(n + e)$
插入边	$O(1)$	$O(1)$
删除边	$O(1)$	$O(n)$

- 稠密图（ e 比较大）适于采用邻接矩阵；稀疏图适于采用邻接表



提纲

- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树



图的遍历

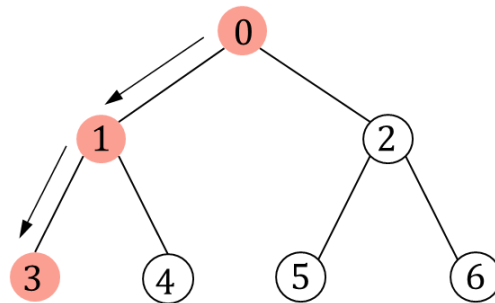
- 从连通图中某一顶点出发，沿着一些边访问图中所有的顶点，且使每个顶点仅被访问一次，就叫做图的遍历
 - 一个顶点可能会多次到达，但访问仅一次
 - 为了避免重复访问，可设置一个标志顶点是否被访问过的辅助数组visited[]
- 基本方法（思考与线性结构、树结构的区别）
 - 深度优先遍历（搜索） (depth-first traversal (search, DFS))
 - 广度优先遍历（搜索） (breadth-first traversal (search, BFS))
 - 无向图和有向图均适用



图的深度优先遍历

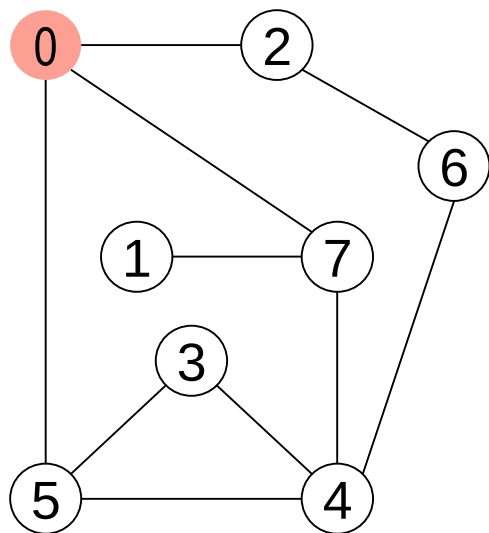
1. 访问图中某一起始顶点 v ;
2. 由 v 出发, 访问它的任一邻接顶点 w_1 ;
3. 从 w_1 出发, 访问与 w_1 邻接但还没有访问过的顶点 w_2 ;
4. 重复步骤3, 直至到达所有的邻接顶点都被访问过的顶点为止;
5. 退回一步, 到前一次刚访问过的顶点, 看是否还有其它没有被访问的邻接顶点; 如果有, 访问此顶点;
6. 重复上述过程, 直到连通图中所有顶点都被访问。

- 树的前序遍历是深度优先的
- 深度优先遍历可以用递归来实现

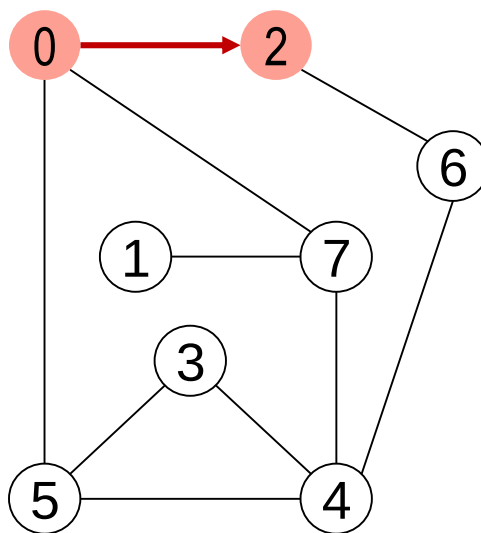




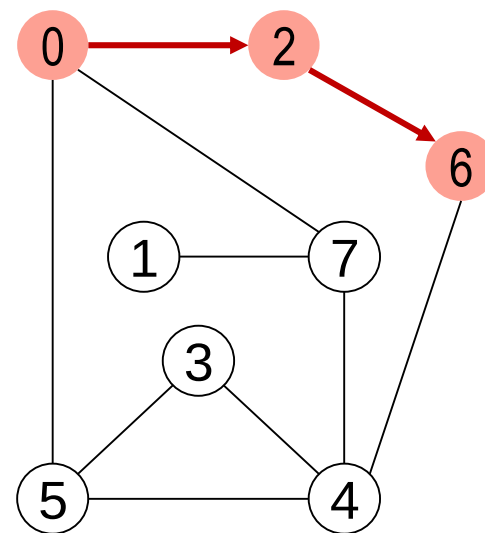
图的深度优先遍历 - 举例



0



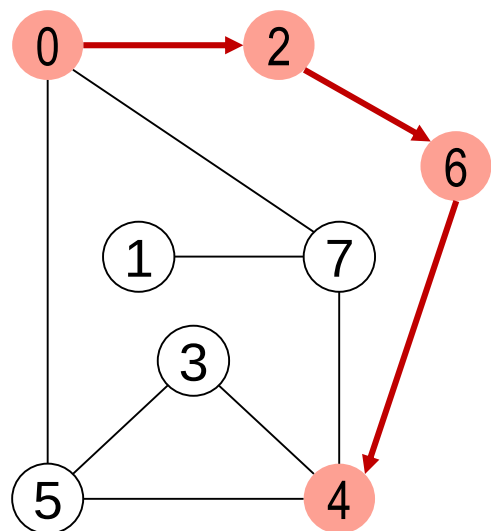
0-2



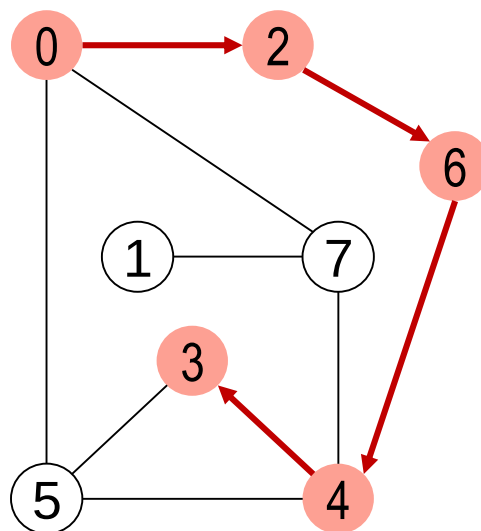
0-2-6



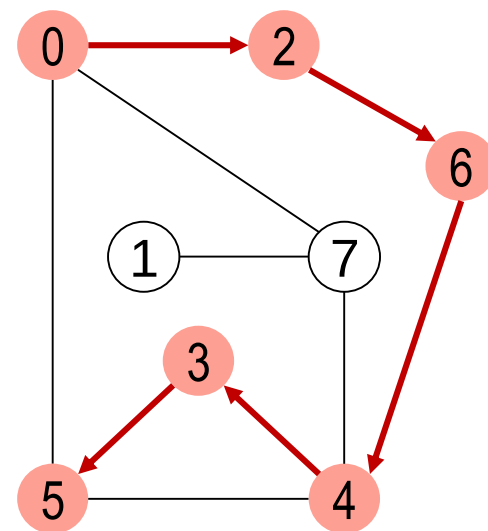
图的深度优先遍历 - 举例



0-2-6-4

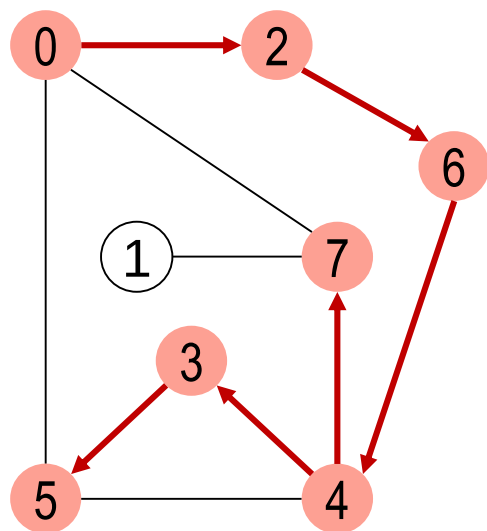


0-2-6-4-3

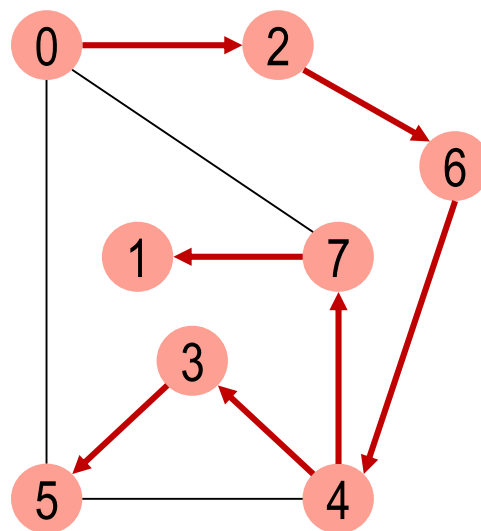


0-2-6-4-3-5

图的深度优先遍历 - 举例



0-2-6-4-3-5-7



0-2-6-4-3-5-7-1



霍去病：骠骑将军

长途奔袭、深入大漠

直取祁连山，封狼居胥山

深度优先遍历序列不唯一：与出发顶点有关，与选择邻接顶点的顺序有关



图的深度优先遍历 - 递归实现

```
void GRAPH::DFS(int v){
```

```
    visit(v);
```

//访问当前结点

```
    visited[v] = true;
```

//设置访问标志

```
    for(int t = FirstAdjVex(v); t != -1; t = NextAdjVex(v,t)) {
```

终止条件

//依次访问顶点v的邻接顶点

```
        if(visited[t] == false) DFS(t);
```

//递归遍历邻接结点

```
    }
```

大规模→小规模问题

```
}
```

对于邻接矩阵和邻接表的表示均适用，FirstAdjVex() 和NextAdjVex() 内部不同而已



图的深度优先遍历 - 计算复杂度

时间复杂度：

➤ 邻接表

- 共有 $2e$ 个边结点(无向图)，遍历时间复杂度为 $O(n + 2e)$
- 用邻接表表示图，沿链可以找到某个顶点 v 的所有邻接顶点。所以扫描边的时间为 $O(2e)$ ，并且对所有顶点访问1次，所以遍历图的时间复杂度为 $O(n + 2e)$

➤ 邻接矩阵

- 查找每个顶点的所有边，时间复杂度为 $O(n)$
- 遍历图中所有顶点的时间复杂度为 $O(n^2)$

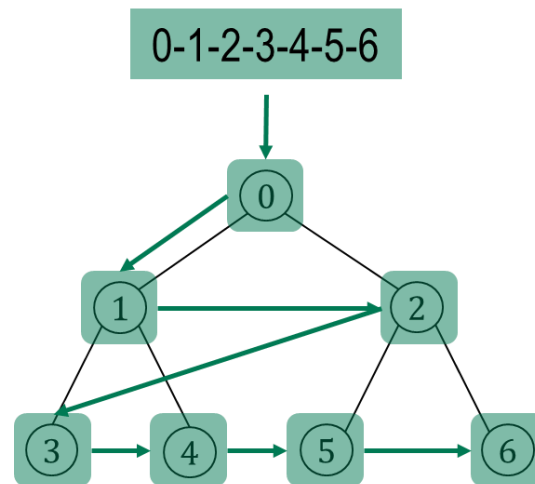
空间复杂度：主要是递归调用对递归栈的使用，最多的递归调用次数为 n ，因此，两种存储结构的时空复杂度都为 $O(n)$



图的广度优先遍历

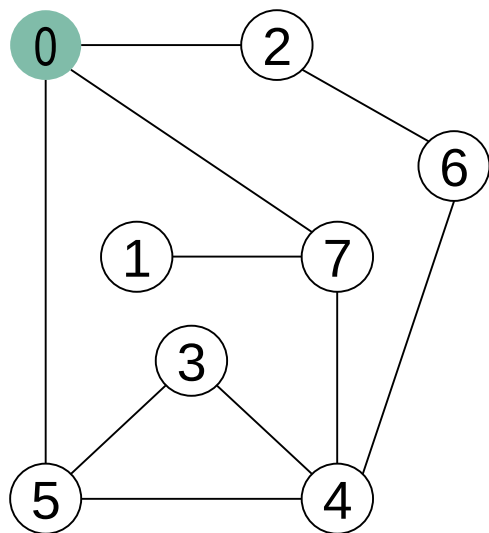
1. 访问遍历的起始顶点 v ;
2. 由 v 出发, 依次访问 v 的各个未曾被访问过的邻接顶点 $w_1, w_2, \dots, w_t$;
3. 然后再顺序访问 $w_1, w_2, \dots, w_t$ 的所有还未被访问过的邻接顶点;
4. 如此做下去, 直到图中所有顶点都被访问到为止。

- 类似于树的层序遍历
- 广度优先遍历是一种分层的搜索过程,
不能用递归实现

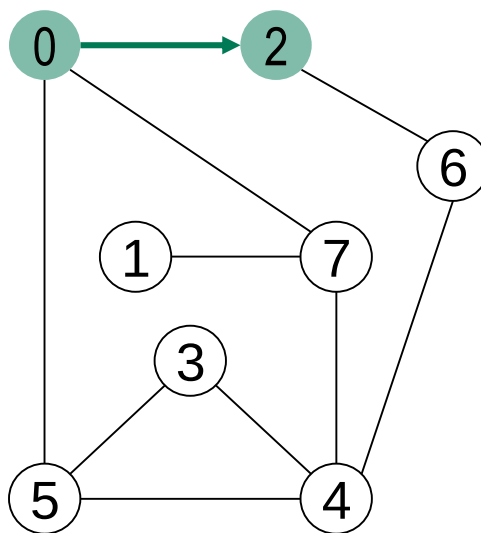




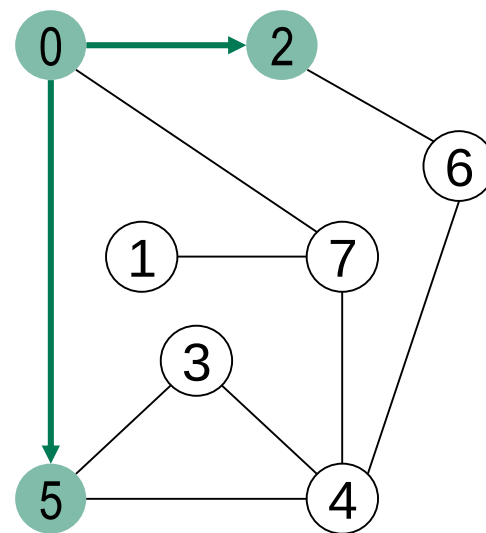
图的广度优先遍历 - 举例



0



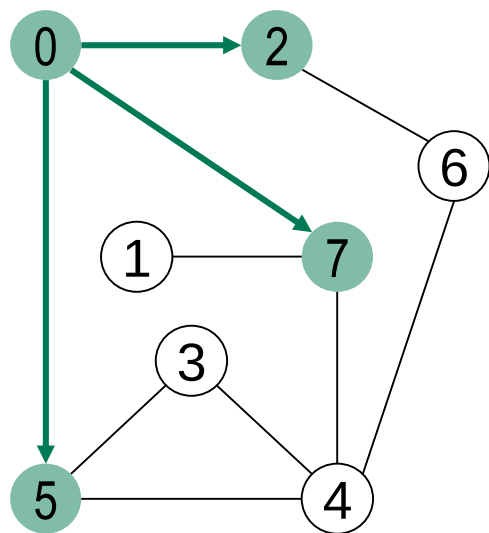
0-2



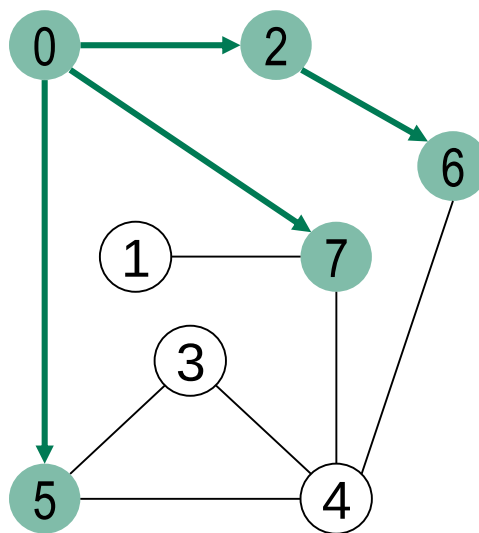
0-2
-5



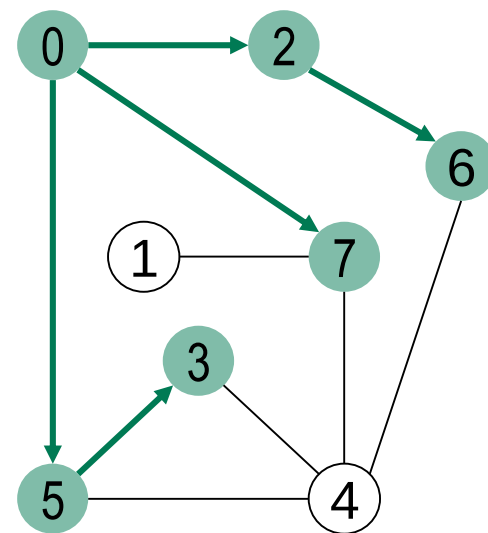
图的广度优先遍历 - 举例



0-2
-5
-7

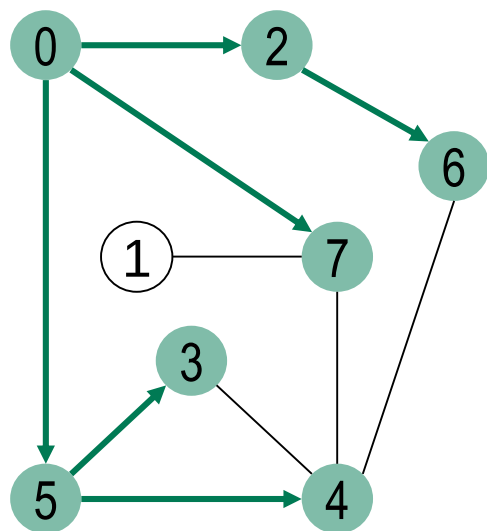


0-2-6
-5
-7

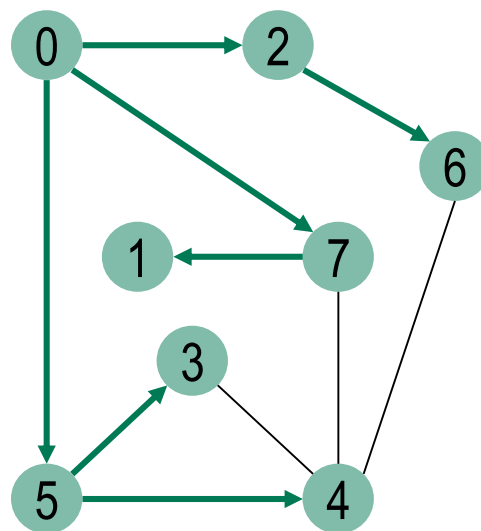


0-2-6
-5-3
-7

图的广度优先遍历 - 举例



0-2-6
-5-3
-4
-7



0-2-6
-5-3
-4
-7-1



卫青：大将军

以武刚车结阵，围追堵杀
力战破敌，烧其积粟而还



图的广度优先遍历 - 实现

```
void BFS(Graph *graph, int v) {  
    int i, j;  
    CirQueue queue;  
    Init_CirQueue(&queue, graph->Vcnt);           //初始化循环队列  
    graph->visited[v] = 1;                          //置访问标志  
    En_CirQueue(&queue, v);                        //顶点入队列  
    printf("%d ", v);                              //访问起始顶点  
    while (!CirQueue_Empty(&queue)) {  
        i = Out_CirQueue (&queue);                //队头元素出队
```



图的广度优先遍历 - 实现

```
for (j=FirstAdjVex(graph, i); j != i; j = NextAdjVex(graph, i, j)) {
```

//依次访问邻接顶点

```
    if (graph->visited[j]==0) {
```

```
        graph->visited[j] = 1;
```

```
        En_CirQueue(&queue, j);
```

//刚访问过的元素入队

```
        printf("%d ", j);
```

```
    }
```

```
}
```

```
Clear_CirQueue(&queue);
```

//销毁队列

```
}
```

- 遍历中各顶点被访问顺序由此顶点到遍历出发顶点的路径长度决定
- 对于邻接矩阵和邻接表的表示均适用



图的广度优先遍历 - 计算复杂度

时间复杂度与深度优先遍历类似

- 邻接表复杂度为 $O(n + 2e)$
- 邻接矩阵复杂度为 $O(n^2)$

空间复杂度：辅助空间主要来自队列

- 如果遍历过程中对所有未访问的顶点都进行入队操作，则队列中顶点数最多与图的边数相当，为 $O(e)$
- 如果遍历过程中加入队与否的标志，已经入队则不再重复入队，则队列中的顶点数最多与图的顶点数相当，为 $O(n)$



图遍历的应用 - 求简单路径

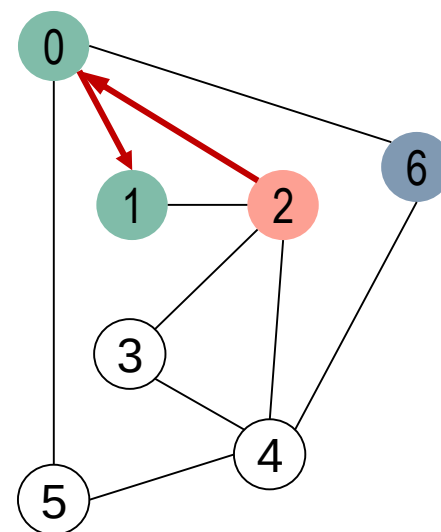
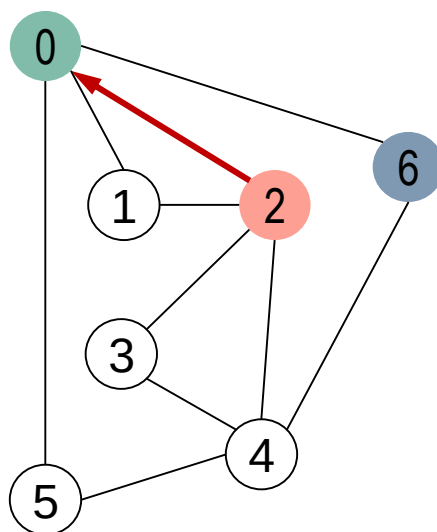
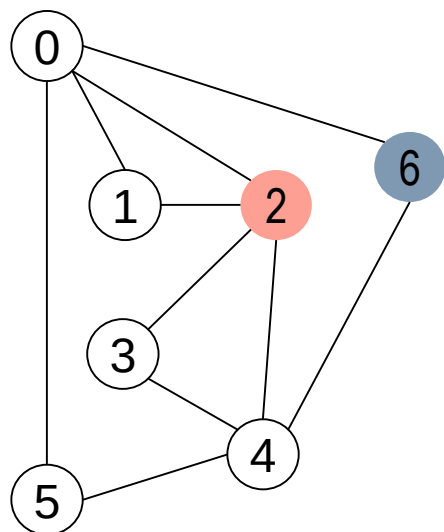
寻找图中从顶点 v 到顶点 w 的简单路径（没有重复出现的顶点）

- 从顶点 v 出发做深度优先遍历，如果遇到顶点 w ，回溯得到从 v 到 w 的一条路径
- 如何保证是简单路径？——维护一条记录，依次记录深度优先遍历过程中访问过的顶点，在遍历过程中，如果某个顶点的所有邻接顶点都被访问过，仍然未能到达目标顶点，则将此顶点从路径中删除，这样到达目标结点后，就可以得到一条简单路径



图遍历的应用 - 求简单路径举例

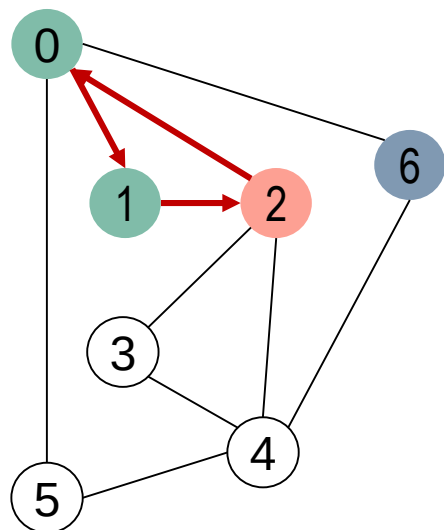
求顶点2到6的简单路径



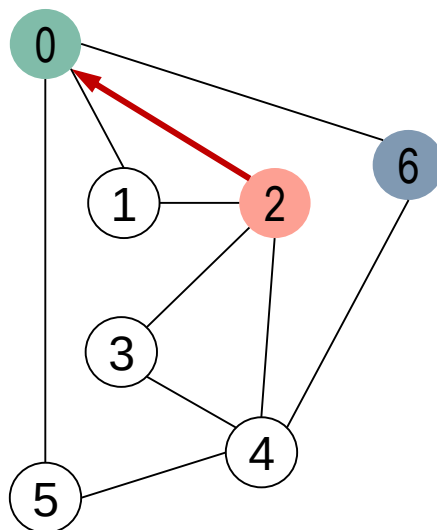


图遍历的应用 - 求简单路径举例

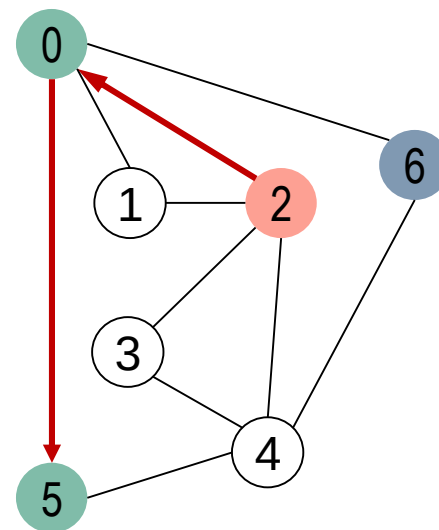
求顶点2到6的简单路径



2-0-1-0
-1-2



2-0-1-0
-1-2
-0-2

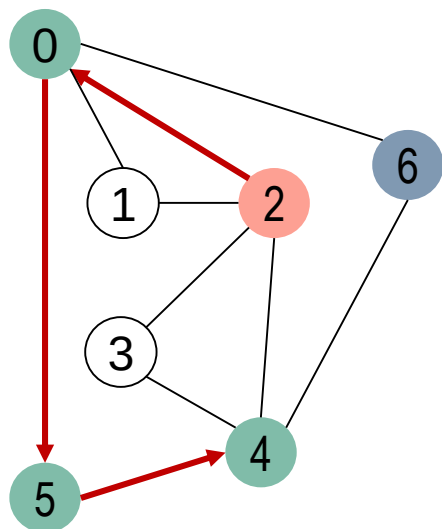


2-0-1
-0-2
-0-5

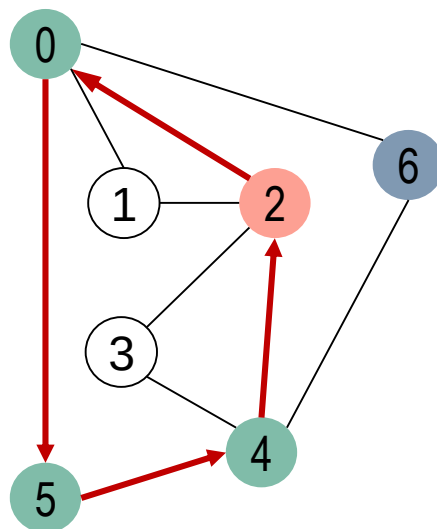


图遍历的应用 - 求简单路径举例

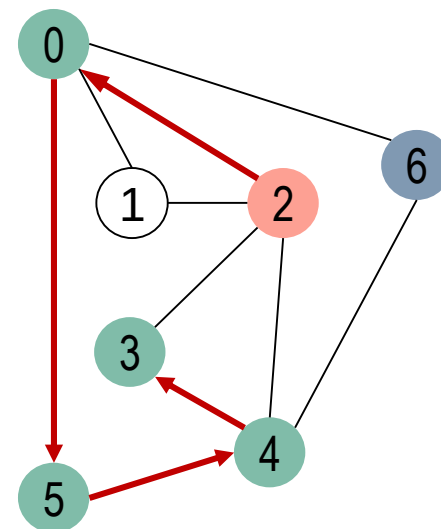
求顶点2到6的简单路径



2-0-1
-0-2
-0-5-0
5-4



2-0-5-0
5-4-2

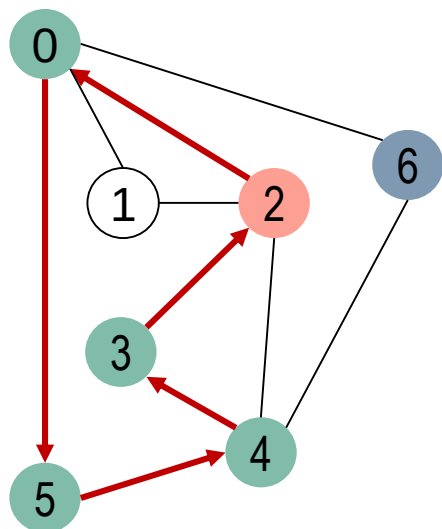


2-0-5-0
5-4-2
4-3

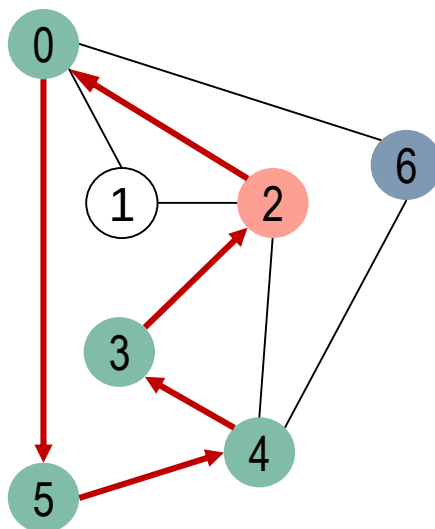


图遍历的应用 - 求简单路径举例

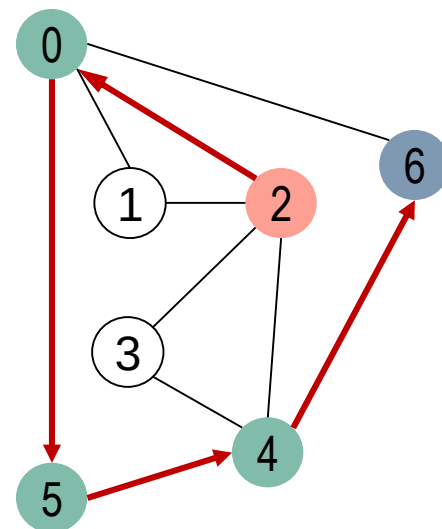
求顶点2到6的简单路径



2-0-5-0
5-4-2
4-3-2



2-0-5-0
5-4-2
4-3-2
3-4

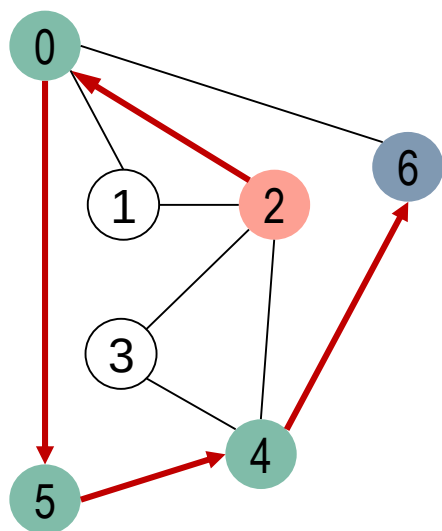


2-0-5-0
5-4-2
4-6



图遍历的应用 - 求简单路径举例

求顶点2到6的简单路径



2-0
0-1
1-0
1-2
0-2
0-5
5-0
5-4
4-2
4-3
3-2
3-4
4-6

删除顶点1

删除顶点3

2-0-5-4-6

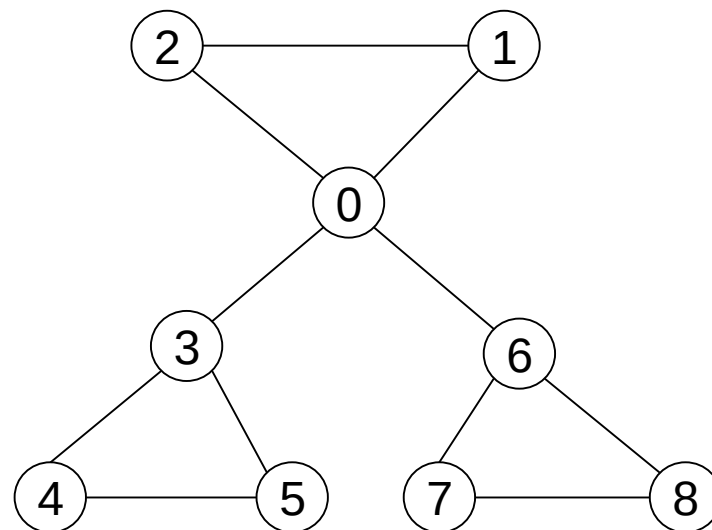


图遍历的应用 - 求无权图最短路径

- 求图中从顶点 v 到顶点 w 的最短路径
- 利用广度优先搜索实现：先访问到出发顶点路径长度较短的顶点，后访问到出发顶点路径长度较长的顶点

2							
2	0	1					
2	0	1	3	6			
2	0	1	3	6	4	5	
2	0	1	3	6	4	5	

求顶点2到顶点4的最短路径



- 2-0-3-4，路径长度为3
- 实现中被访问过的顶点仍保留队列中，从而可以回溯得到路径

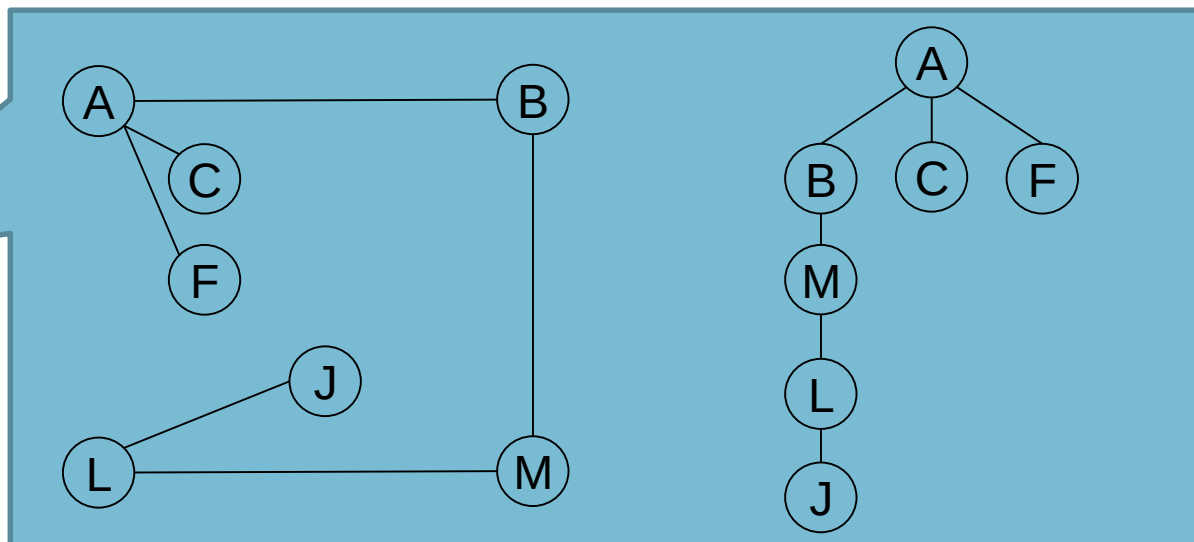
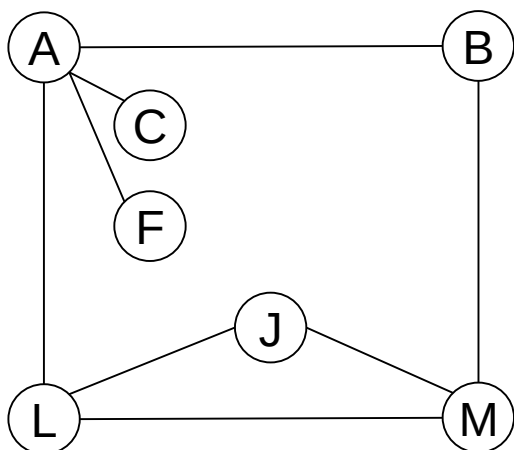


提纲

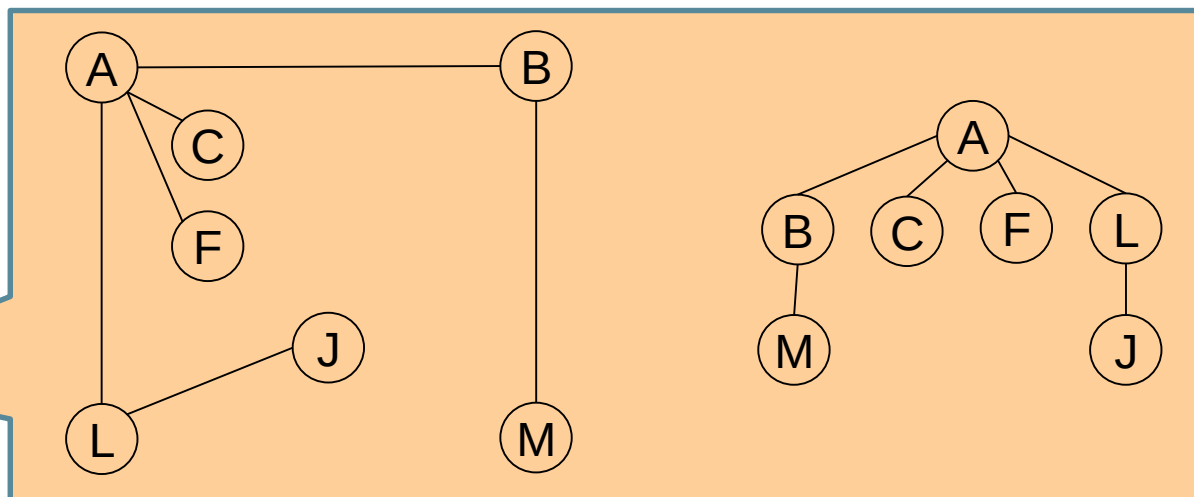
- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树

图中有树

深度优先遍历

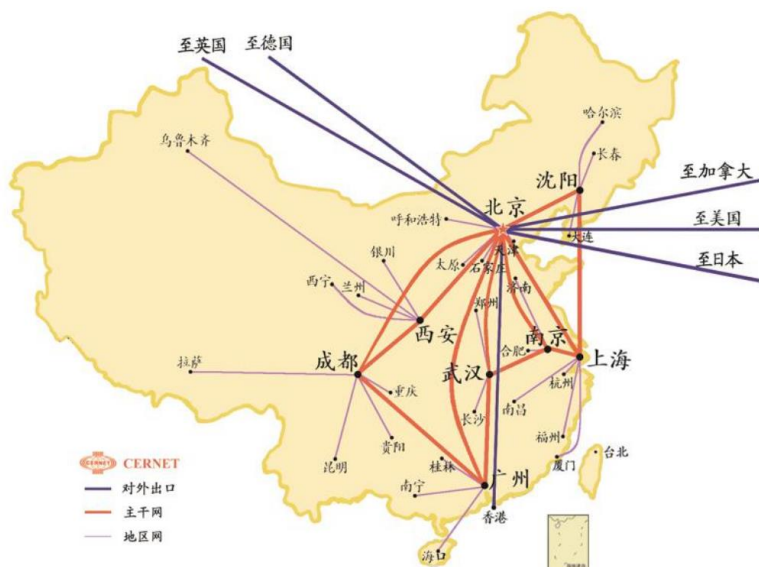


广度优先遍历



最小生成树的意义

- 网络（带权无向图）建设的成本最小化问题
- 权值可以是建设成本或距离等



中国互联网骨干网示意图



北京高速公路分布图



图的生成树

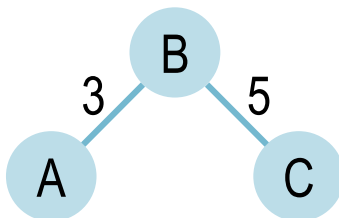
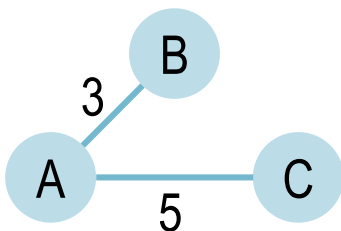
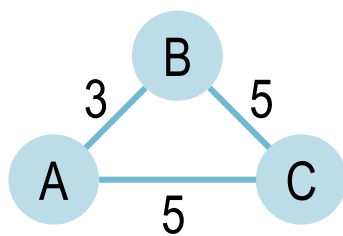
- 最小生成树：找出连通所有顶点的最低成本方案
- 生成树定义：有 n 个顶点的连通图的生成树，是包含图中全部 n 个顶点，但只有 $n - 1$ 条边的连通子图
 - 删掉一条边，不再连通
 - 添加一条边，构成回路
 - 一个极小（边数最少）连通子图
- 构造生成树的准则
 - 必须使用该网络中的边来构造生成树
 - 必须用且仅用 $n - 1$ 条边来联结网络中的 n 个顶点
 - 不能使用产生回路的边



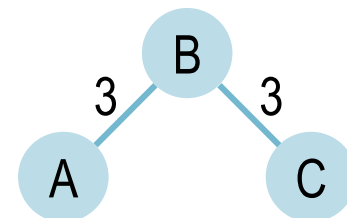
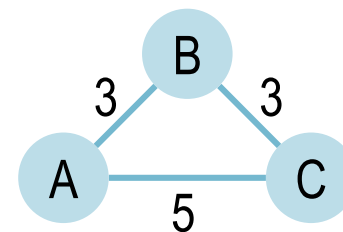
图的最小生成树

- 对于加权无向连通图，每条边都被赋予一个权值。最小生成树 (Minimum Spanning Tree, MST) 是一棵生成树，并且其权值(所有边的权值之和)不会大于其它任何生成树的权值
- 如果边有相同的权值，那么最小生成树并不一定是唯一的，但并不是只要有相同权值的边，最小生成树就一定不唯一

不唯一的例子



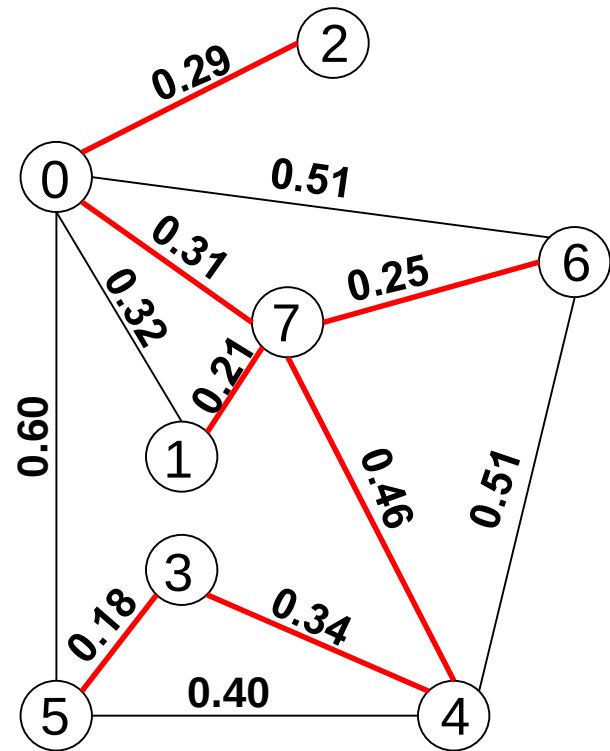
唯一的例子





图的最小生成树

- 使用该网络中的边来构造生成树
- 用 $n-1$ 条边来联结网络中的 n 个顶点
- 不能使用产生回路的边
- 权值最小
- 搜索所有可能的 $n-1$ 条边的组合，找权值最小的生成树（蛮力法）
- 在图中删减边（删权值大的）得到一颗最小生成树
- 在图的顶点、边集合中，从无到有，长出（逐顶点/逐边）一颗最小生成树

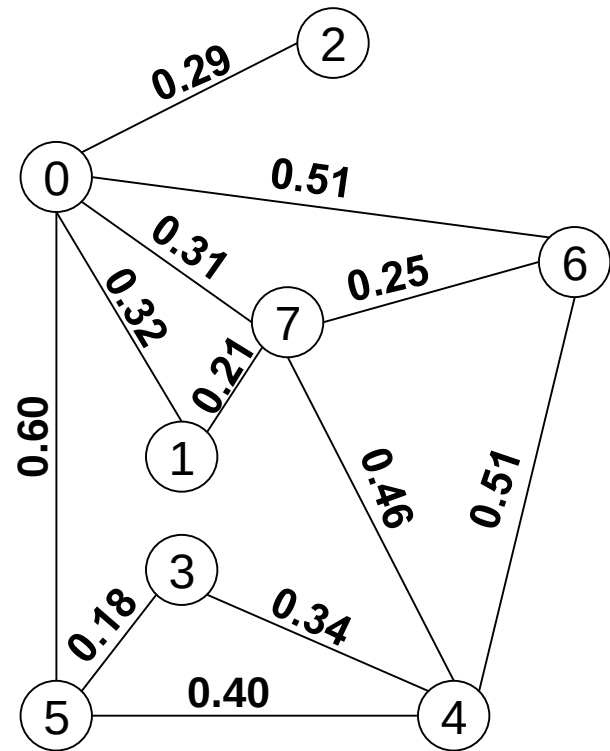


Prim算法、Kruskal算法



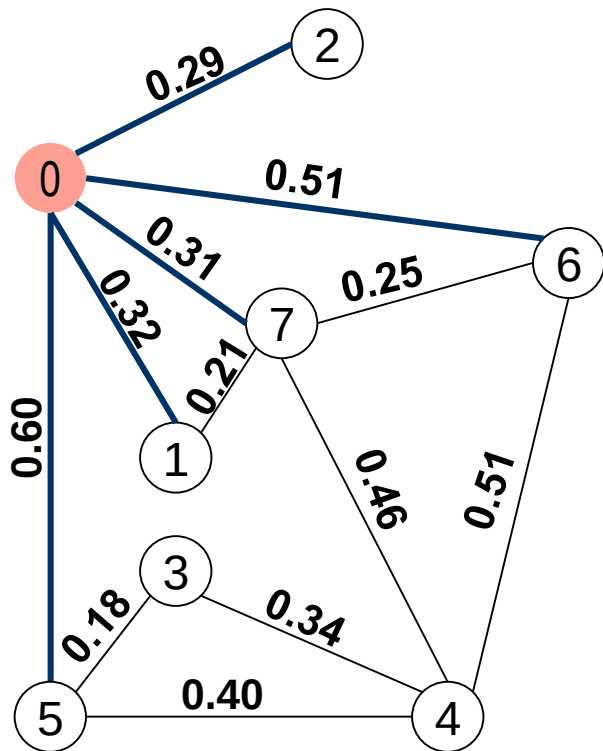
最小生成树的构造算法 - Prim算法

- 初始 MST 是一个空集
- 首先从图 G 中任取一个顶点 a 加入 MST ,
 $MST = \{a\}$, 找出当前非 MST 的顶点与当前 MST 直接相连的所有边
- 取出最短边, 把它和对应的非 MST 顶点 b 加入 MST , $MST = \{a, b\}$, 更新边序列
- 重复上述过程, 直到 MST 包括 G 中所有顶点

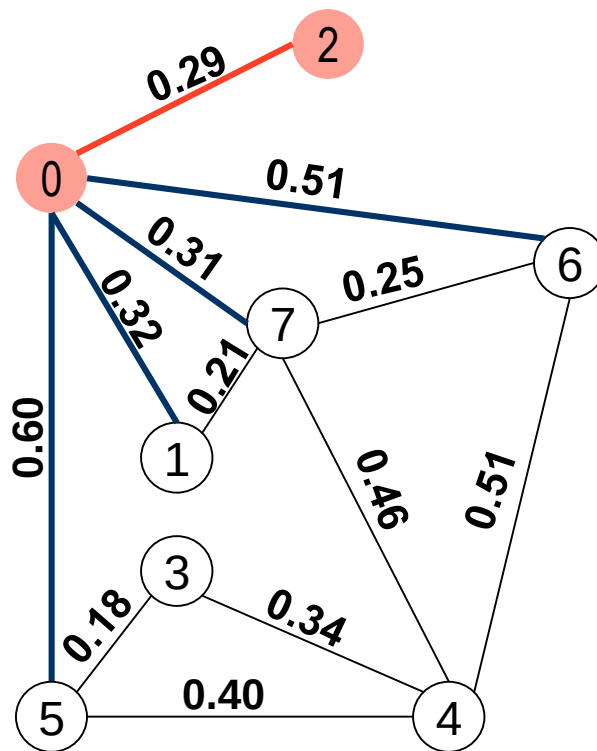




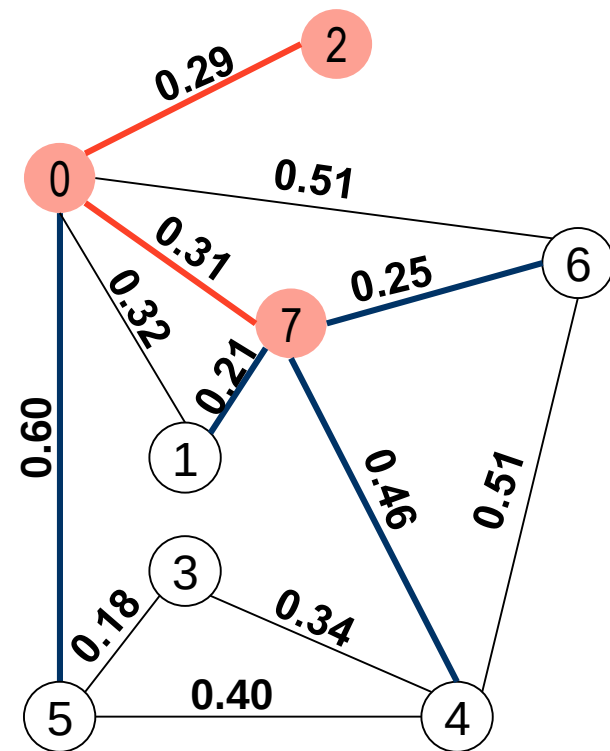
最小生成树的构造算法 - Prim算法



MST = {0}



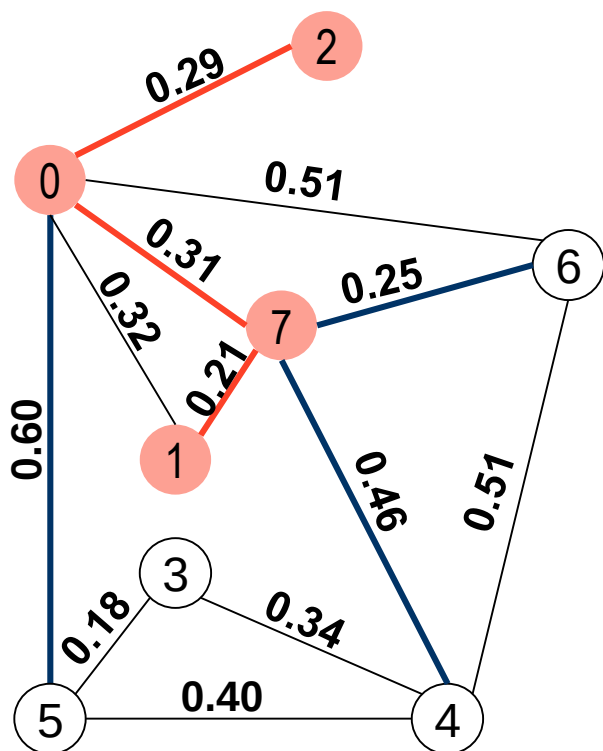
MST = {0, 2}



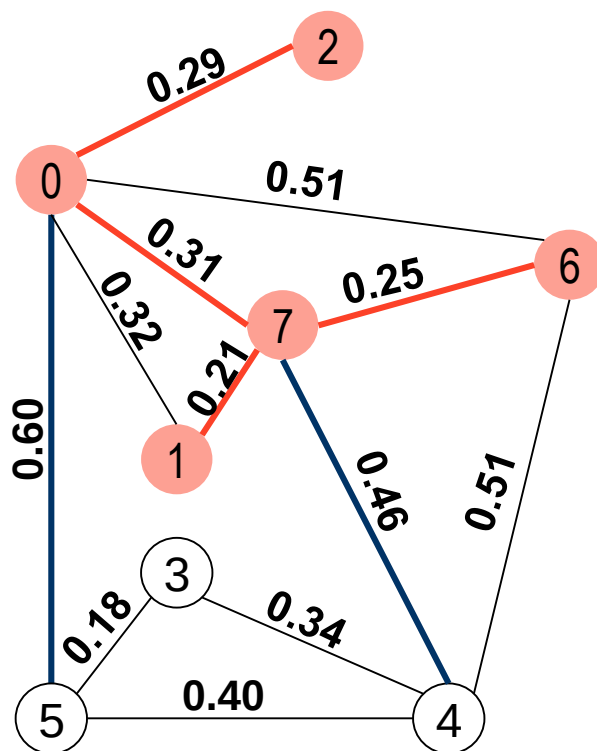
MST = {0, 2, 7}



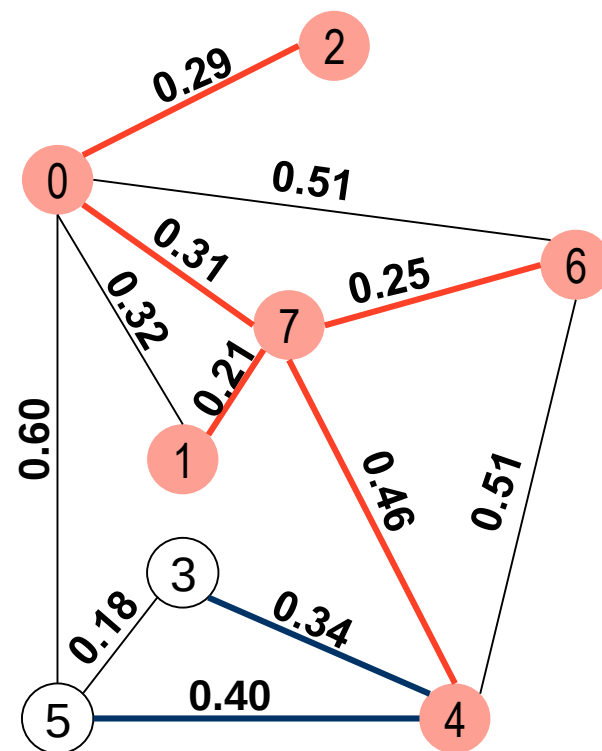
最小生成树的构造算法 - Prim算法



MST = {0, 2, 7, 1}



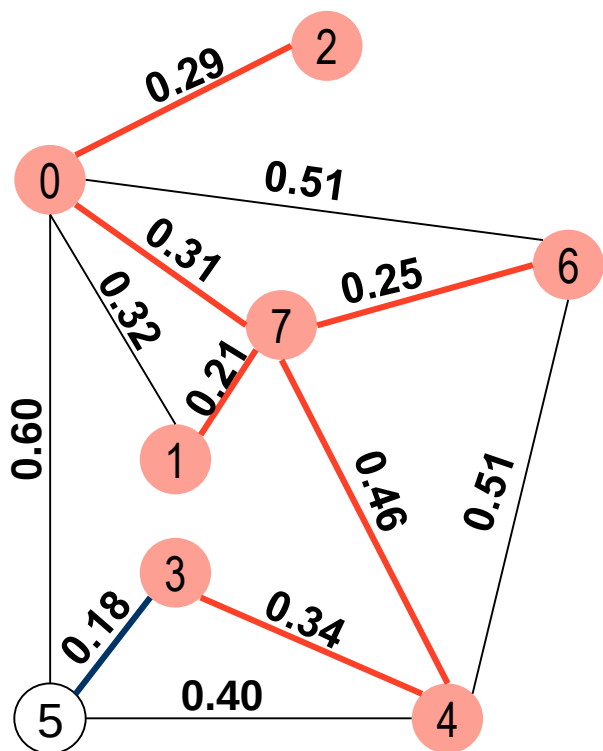
MST = {0, 2, 7, 1, 6}



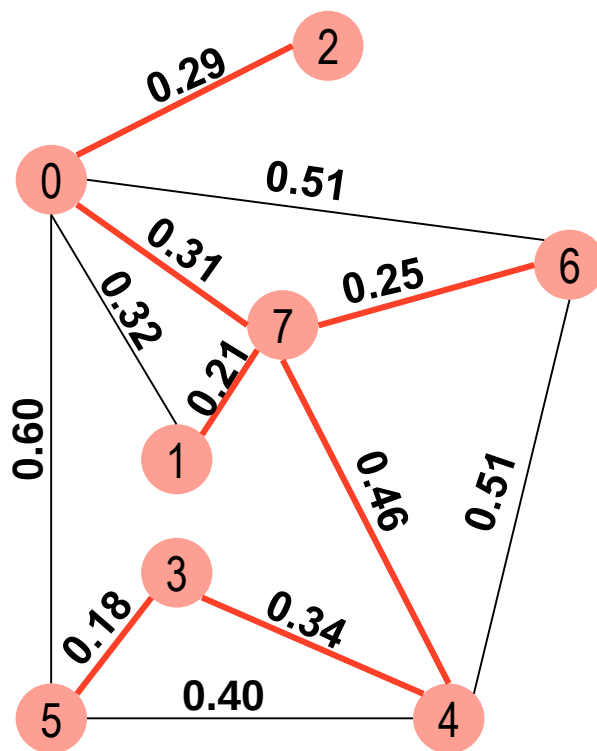
MST = {0, 2, 7, 1, 6, 4}



最小生成树的构造算法 - Prim算法



$MST = \{0, 2, 7, 1, 6, 4, 3\}$



$MST = \{0, 2, 7, 1, 6, 4, 3, 5\}$



最小生成树的构造算法 - Prim算法

- Prim算法将图分成两部分， MST 和非 MST ，每次通过寻找连接两部分的最短边得到距离 MST 最近的顶点，并把最短边和这个顶点加入 MST ，直到 MST 包含了所有顶点
- 程序实现（参考教材p141的Prim函数）思路：
 1. 建立辅助数组，记录各个非 MST 顶点到 MST 的最短距离
 2. 向 MST 加入距离最小的顶点和对应边（参考教材p141的minEdge函数）
 3. 更新辅助数组
 4. 重复步骤2，3，直至非 MST 中无顶点



最小生成树的构造算法 - Prim算法

- Prim算法的时间复杂度是 $O(n^2)$ ，因此对于稠密图来说是线性的
 - MST每次加入一个顶点： $O(n)$
 - 每次加入第一个顶点前，遍历更新各个非MST顶点到MST的最短距离（更新辅助数组），并找出最小距离： $O(n)$
- 对于稠密图，Prim的邻接矩阵实现是首选方法
- 请同学们思考：
 - Prim算法本质上是贪婪算法，为何可以保证是最小生成树？
 - Prim算法与初始顶点选择有关系吗？

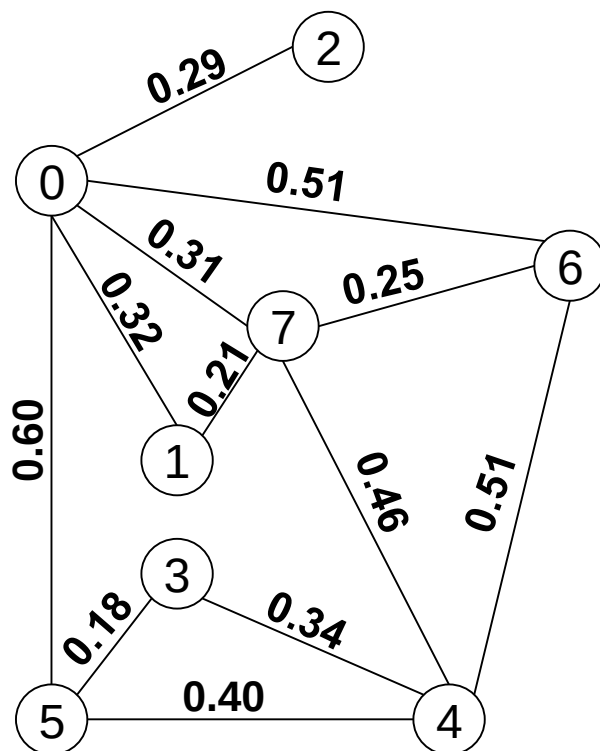


最小生成树的构造算法 - Kruskal算法

- 对图中所有边按权值进行排序
- 令 MST 的边集为空
- 从图中取出权值最小的一条边，如果这条边与 MST 中的边能构成环，则舍弃；否则，将这条边加入 MST
- 重复上述过程，直至将 $n - 1$ 条边加入 MST



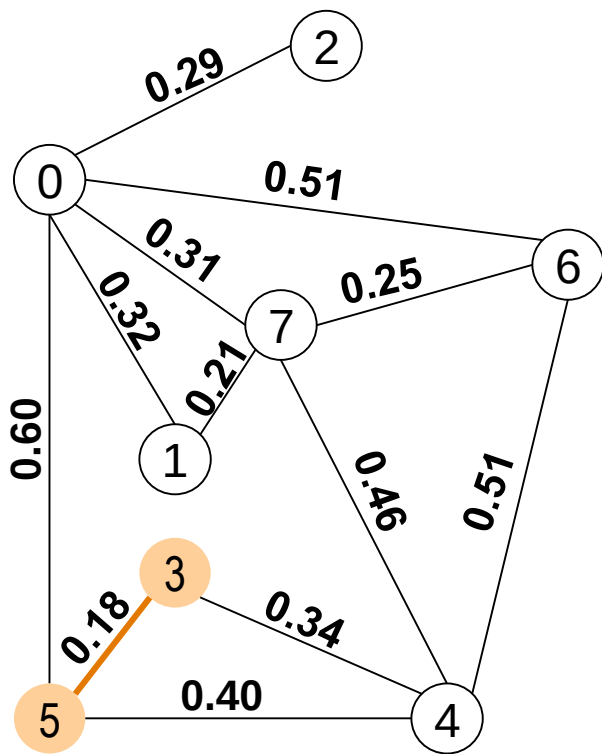
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



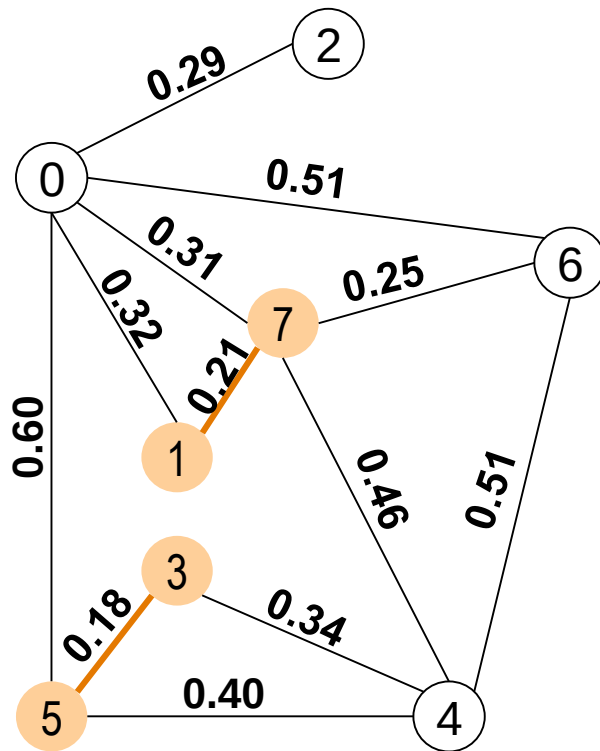
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



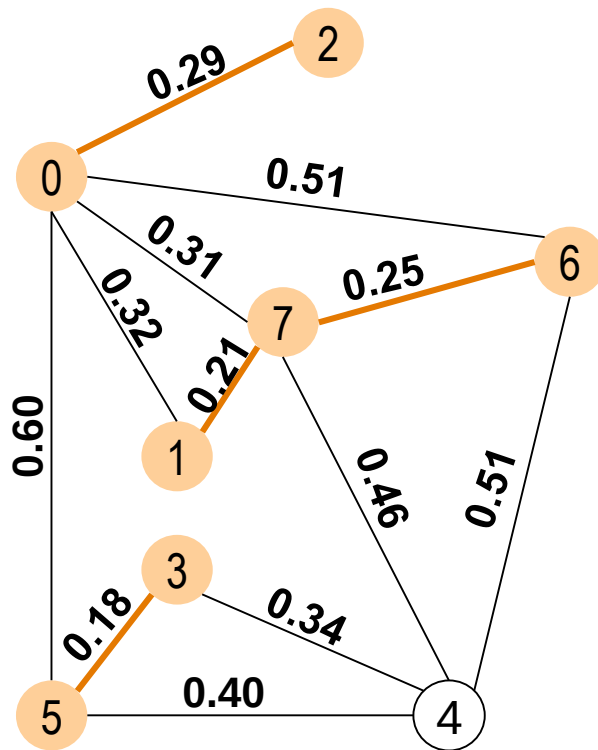
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



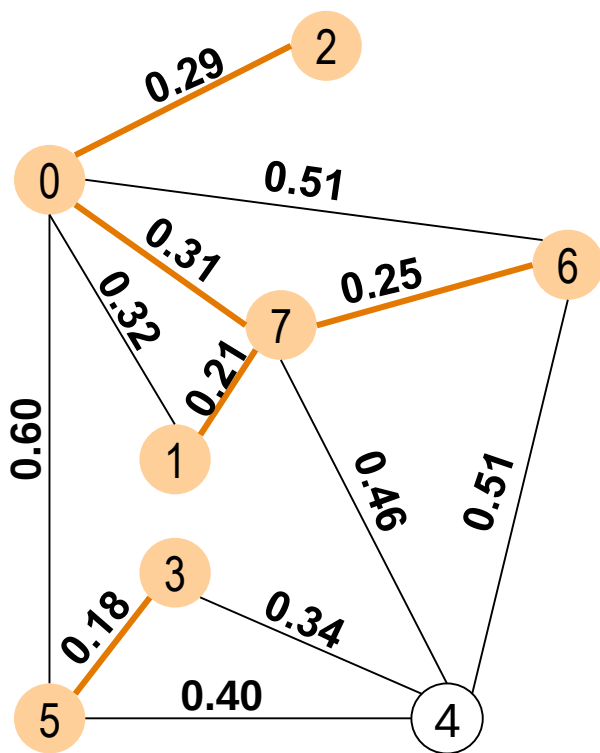
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



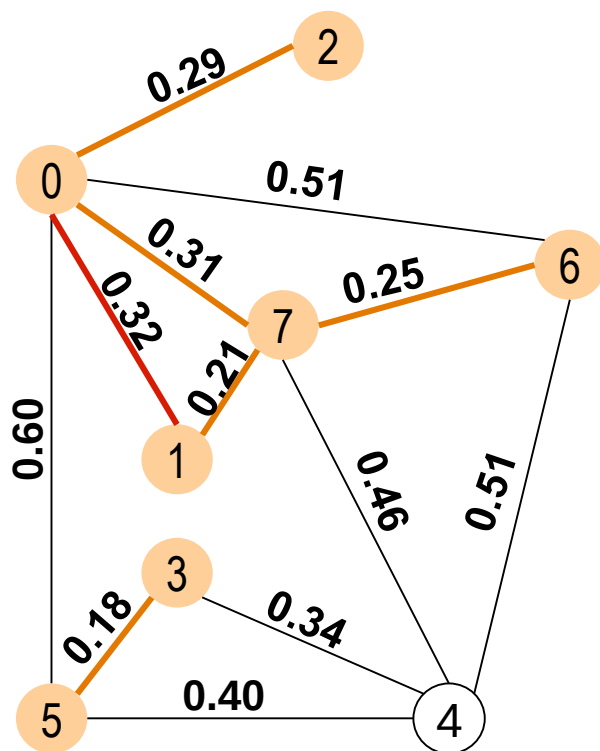
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



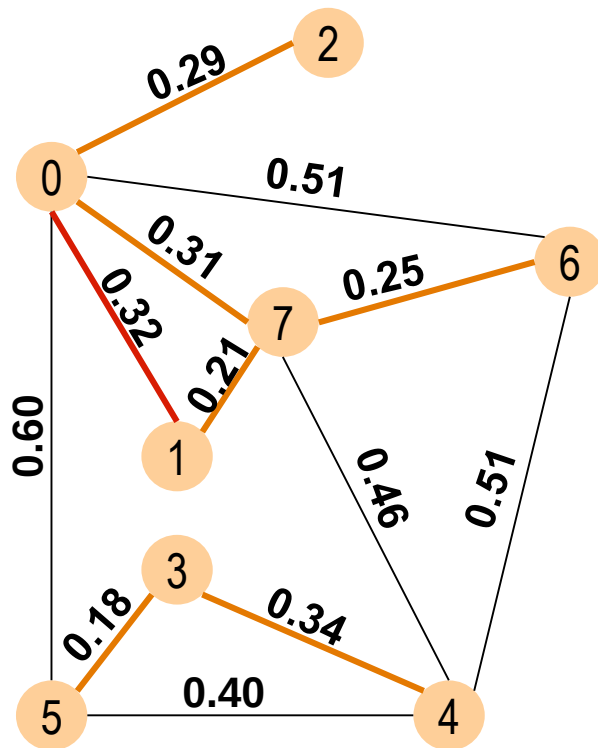
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



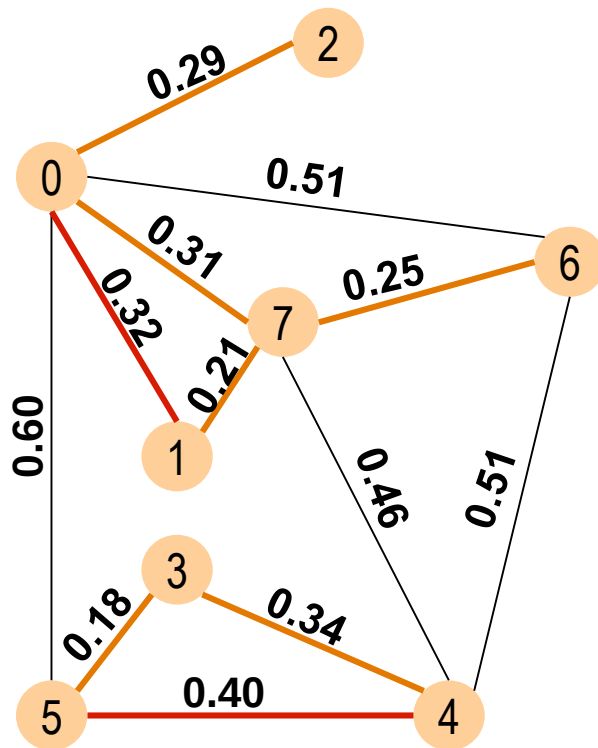
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



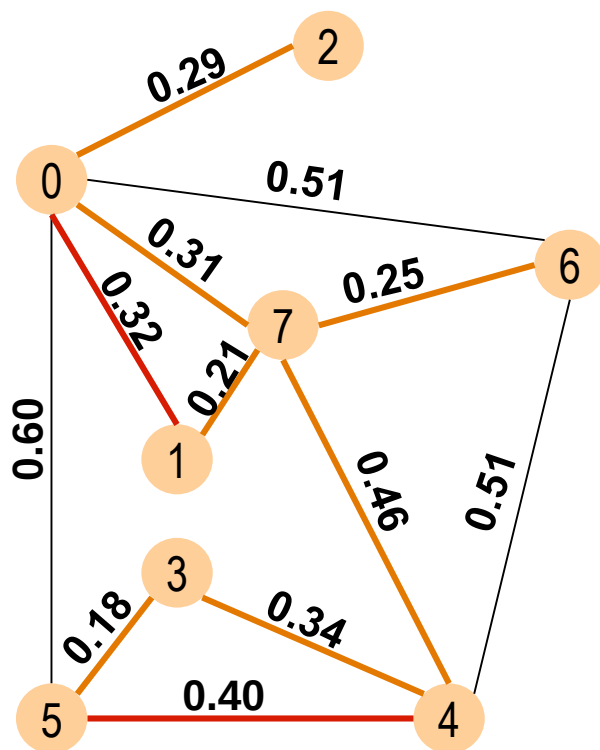
最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60



最小生成树的构造算法 - Kruskal算法



5-3	0.18
7-1	0.21
7-6	0.25
0-2	0.29
7-0	0.31
0-1	0.32
4-3	0.34
5-4	0.40
7-4	0.46
0-6	0.51
6-4	0.51
0-5	0.60

共7条边

最小生成树的构造算法 - Kruskal算法



- Kruskal算法本质上也是贪婪算法：每次向 MST 中加入权值最小的边，并保证 MST 中不构成回路
- 在Kruskal算法中，如果找到了 $n - 1$ 条边，则得到最小生成树，如果检查了所有边，而未找到 $n - 1$ 条边，则此图是非连通的
- 算法复杂度
 - 对边的排序，采用高级排序方法，时间复杂度可以达到 $O(e \log e)$
 - 环的判断：维护一个连通关系的等价类集合（教材p143程序示例中的顶点连通子集），新增边的两个顶点如果在集合里，则加入后会构成环，否则合并入集合，其时间复杂度近乎线性 $O(n)$
 - Kruskal算法的时间复杂度为 $O(e \log e)$ ，适用于稀疏图



本讲小结

- 图的基本概念
- 图的存储结构：稠密图适于采用邻接矩阵；稀疏图适于采用邻接表
- 图的遍历：深度优先遍历 vs. 广度优先遍历
- 最小生成树

	优选存储结构	优选最小生成树构造方法	复杂度
稠密图	邻接矩阵	Prim算法	$O(n^2)$
稀疏图	邻接表	Kruskal算法	$O(e \log e)$

加油同学们!

